Universidad de Caldas
Facultad de Inteligencia Artificial e Ingenierías

Análisis y Diseño de Algoritmos — 2026-B

Proyecto semestral

Subproyecto 2 — Reparto de presupuesto y selección de participantes

Primera entrega: formalización y prueba de NP-dureza

Integrante: Juan Esteban Guzmán Henao

Profesora: Luz Enith Guerrero

Grupo: 2

Repositorio: `https://github.com/GuzGuz0208/ADA`

Versión: etiqueta `v0.1-primera-entrega`

Manizales, 6 de noviembre de 2026

Índice

Convenciones del documento **2**

1. Formalización del problema **2**
1.1. Contexto general . . . 2
1.2. Datos comunes y esquema de la instancia . . . 2
1.3. Parte 1 — Reparto óptimo del presupuesto . . . 3
1.4. Parte 2 — Selección de participantes . . . 4
1.5. Supuestos de modelado y alcance de la entrega . . . 6

2. NP-dureza de la selección de participantes **6**
2.1. Problema de origen: Cobertura Máxima . . . 7
2.2. Versión de decisión de la Parte 2 y pertenencia a NP . . . 7
2.3. La transformación φ y su costo . . . 7
2.4. Equivalencia de respuestas . . . 8
2.5. Ejemplo trabajado a mano . . . 9
2.6. Refuerzo: la dureza se mantiene en la subfamilia del JSON . . . 10
2.7. Consecuencias y conclusión . . . 11

3. Algoritmos base: optimalidad, costo y garantía **11**
3.1. Parte 1: el voraz por ganancia marginal es óptimo . . . 11
3.1.1. Costo del Algoritmo 2 . . . 13
3.2. Parte 2: voraz de mayor ganancia marginal y su costo . . . 14
3.3. Mejor caso, peor caso y caso promedio . . . 15
3.4. Cota $1 - 1/e$ del voraz (modo `k`) . . . 21

4. Ejecución del caso de la profesora **22**
4.1. Resultado por ronda . . . 22
4.2. Traza resumida de la ronda 1 . . . 23
4.3. Verificación de restricciones . . . 24
4.4. Certificados y comparación con el exacto . . . 24
4.5. Tiempos de ejecución . . . 25
4.6. Advertencia de saturación (R-02) . . . 25

Convenciones del documento

- **Demostrado por nosotros** vs. **citado**: cada resultado indica si la prueba está completa en este informe o si se toma de la bibliografía. Las referencias que no se han podido confirmar con la fuente original llevan **[VERIFICAR]**.
- **Cifras del caso de la profesora**: todas provienen de la ejecución de `python -m ada_g2 run ...` con la semilla 20261105 y están en `results/caso_entrada_G2_red_15_nodos/`; la Sección 4 cita el archivo de origen de cada una. Ninguna se escribió a mano.
- El símbolo ■ al final de un párrafo marca el **fin de una demostración** (equivale a "q.e.d.", como en las notas del curso [2]).
- Los supuestos de modelado se identifican como D-xx (registro de decisiones del proyecto, `agent/DECISIONS.md`).
- $\mathbb{N} = \{0, 1, 2, \dots\}$; $[n] = \{1, \dots, n\}$; ln es el logaritmo natural; log sin base indica base 2 en los costos.

1. Formalización del problema

1.1. Contexto general

Una red de N nodos entrena colaborativamente un modelo durante T rondas sin compartir datos crudos. Cada actualización se protege con ruido calibrado: más ruido protege más pero reduce la utilidad de la actualización. Cada nodo dispone de un presupuesto total de "ruido tolerable" para todo el proyecto y, en cada ronda, el sistema toma dos decisiones [1]:

(a) **qué nodos participan** (Parte 2, selección de participantes), y

(b) **cuánto presupuesto recibe cada participante** (Parte 1, reparto del presupuesto).

Siguiendo la Figura 3.1 del enunciado, las dos partes se resuelven *en cadena* dentro de cada ronda: primero se elige el conjunto S_t y luego se reparte el presupuesto de la ronda entre los nodos de S_t (D-08). Cada parte es un problema de optimización distinto, con su propio tamaño de entrada, y se formaliza por separado.

1.2. Datos comunes y esquema de la instancia

La profesora entrega instancias en JSON con el esquema de `entrada_G2_red_15_nodos.json` y puede ejecutar *otras* instancias con el mismo esquema; por eso ningún valor de la instancia se fija en el código. El esquema contiene:

- `parametros`: número de nodos N, número de rondas T, cupo por ronda k, presupuesto de ruido por ronda B, presupuesto de costos de participación por ronda L.
- `nodos`: para cada nodo i, `coeficiente_utilidad` $a_i > 0$, `presupuesto_total_nodo` $R_i \in \mathbb{N}$ y `costo_participacion` $c_i \in \mathbb{N}$.
- `grafo`: aristas $\{u, v\}$ con `peso_influencia` $w_{uv} \in [0, 1]$.

En la instancia de la profesora, $N = 15$, $T = 3$, $k = 5$, $B = 30$, $L = 24$ y el grafo trae $105 = \binom{15}{2}$ aristas, cada par una sola vez. Los resultados obtenidos sobre esa instancia, producidos por el código con la semilla 20261105, se reportan en la Sección 4.

Lo que el enunciado no fija. Ningún documento del Subproyecto 2 define la forma de u_i, la forma de f ni la probabilidad de falla q; el enunciado pide "precisar τ para la función de utilidad que el equipo implemente". Las elecciones de modelado están en la Tabla 1 (D-01, D-02, D-07).

1.3. Parte 1 — Reparto óptimo del presupuesto

Contexto y decisión. Dado el conjunto de nodos que ya participan en la ronda, se decide cuántas unidades enteras de presupuesto de ruido recibe cada uno para maximizar la utilidad total de la ronda.

Función de utilidad (D-01). Para cada nodo i,

$$u_i(x) = a_i \ln(1 + x), \qquad x \in \mathbb{N},\ a_i > 0,$$

con ganancia marginal

$$g_i(x) = u_i(x + 1) - u_i(x) = a_i \ln \frac{x + 2}{x + 1} = a_i \ln\Big(1 + \frac{1}{x + 1}\Big).$$

Lema 1.1 (u_i es creciente y cóncava sobre $\mathbb{N}$; demostrado por nosotros)**.** *Si $a_i > 0$, entonces $g_i(x) > 0$ para todo $x \in \mathbb{N}$ y g_i es estrictamente decreciente. En consecuencia u_i es estrictamente creciente y cóncava en el sentido discreto del enunciado (cada unidad adicional aporta una ganancia no mayor que la anterior), y $u_i(0) = 0$.*

Demostración. (1) Sea $z(x) = 1/(x+1)$. Para $x \in \mathbb{N}$, $z(x) \in (0, 1]$ y z es estrictamente decreciente. (2) La función $\ln(1+z)$ es estrictamente creciente en z y vale 0 en $z = 0$; luego $\ln(1+z(x)) > 0$ y $x \mapsto \ln(1 + z(x))$ es estrictamente decreciente (composición de creciente con decreciente). (3) Multiplicar por $a_i > 0$ conserva el signo y el orden, así que $g_i(x) > 0$ y $g_i(x + 1) < g_i(x)$. (4) $g_i > 0$ significa $u_i(x + 1) > u_i(x)$ (creciente) y g_i decreciente es exactamente la concavidad discreta pedida. Por último $u_i(0) = a_i \ln 1 = 0$. ■

Observación 1.2. Si algún $a_i = 0$ (fuera del esquema, que exige $a_i > 0$), $g_i \equiv 0$: u_i sigue siendo cóncava y no decreciente, y el Teorema 3.1 sigue valiendo (ver Observación 3.2). Si $a_i < 0$, u_i es decreciente y el nodo nunca debería recibir presupuesto; el programa debe rechazar esa entrada.

Definición 1.3 (Problema Reparto)**.**

- **Entrada.** Un conjunto S de N nodos participantes (en la cadena, $S = S_t$ y $N = |S_t| \le k$); para cada $i \in S$, el coeficiente $a_i > 0$ y el tope $r_i \in \mathbb{N}$ (presupuesto restante del nodo, D-06: $r_i = R_i$ en la ronda 1 y $r_i = R_i - \sum_{s<t} x_{i,s}$ en la ronda t); el presupuesto de la ronda $B \in \mathbb{N}$.
- **Salida.** Un vector $x = (x_i)_{i \in S} \in \mathbb{N}^N$ (unidades asignadas a cada nodo).
- **Restricciones.** $\sum_{i \in S} x_i \le B$ y $0 \le x_i \le r_i$ para todo $i \in S$.
- **Objetivo.** Maximizar $U(x) = \sum_{i \in S} u_i(x_i)$.

Observación 1.4 (relación con la Definición 3.2.1 del enunciado). El enunciado no incluye topes por nodo. Con $r_i \ge B$ para todo i el tope nunca es activo y se recupera exactamente la Definición 3.2.1. Los topes se añaden porque el JSON trae `presupuesto_total_nodo` y el escenario dice que ese presupuesto es total para las T rondas; en la instancia de la profesora sí llegan a ser activos: en la ronda 3 los topes restantes (4, 8, 8, 4, 6) coinciden con el reparto y suman $B = 30$ (Sección 4, Tabla 4; origen: `results/caso_entrada_G2_red_15_nodos/resultado.json`).

Tamaño de la entrada: (N, B)**.** Crecen dos magnitudes independientes:

- N, el número de nodos a repartir: determina cuántos números (a_i, r_i) se leen y el tamaño del montículo;
- B, el número de unidades a repartir: determina cuántas decisiones unitarias toma el voraz.

No se reduce a una sola variable porque el costo del algoritmo (Sección 3.1.1) depende de ambas de forma distinta: $O(N + B \log N)$. Si se mide en bits, la entrada ocupa $\Theta(N\beta + \log B)$ bits (β = bits por coeficiente y tope), de modo que B entra en la longitud solo con $\log B$ bits; un costo lineal en B es, por tanto, **pseudo-polinomial**: polinomial en el *valor* de B, exponencial en su longitud ($B = 2^{\log B}$). Es el mismo fenómeno que en la mochila 0/1 por programación dinámica [1].

Supuestos específicos. Presupuesto en unidades enteras (D-13); los nodos no interactúan (la utilidad total es separable; el caso con grafo es una variante, P2); aritmética real de costo unitario, incluido ln (en el código, `float64` con tolerancia 10^{-9} en comparaciones de prueba, D-13); desempates por menor índice (D-12).

Alcance. Algoritmo exacto voraz por ganancia marginal (Algoritmo 2), su optimalidad (Teorema 3.1) y su costo. Por exigencia del enunciado, no se usa fuerza bruta ni metaheurísticas; la programación dinámica exacta solo se usa en las pruebas como referencia independiente. El caso continuo (*water-filling*) se discute en la Observación 3.5.

1.4. Parte 2 — Selección de participantes

Contexto y decisión. Entre los nodos candidatos de la ronda se elige un subconjunto de participantes que maximice una utilidad conjunta en la que los aportes se *solapan*: dos nodos muy influyentes sobre los mismos vecinos aportan menos juntos que la suma de sus aportes por separado.

Función de utilidad conjunta (D-02). Familia general: un conjunto de N *candidatos* $V = [N]$, un conjunto de M *objetivos* $[M]$ con pesos $a_j \geq 0$, y una matriz de influencias $W = (w_{ij}) \in [0,1]^{N \times M}$. Para $S \subseteq V$,

$$f(S) = \sum_{j=1}^{M} a_j \Big[1 - \prod_{i \in S} (1 - w_{ij})\Big], \qquad f(\emptyset) = 0.$$

Interpretación: w_{ij} es la probabilidad de que el candidato i "cubra" (influya en) el objetivo j, de forma independiente; $f(S)$ es el peso esperado de objetivos cubiertos. **En el JSON**, candidatos y objetivos son los mismos N nodos ($M = N$), a_j es el `coeficiente_utilidad`, $w_{ij} = w_{ji} =$ `peso_influencia` (grafo no dirigido, D-10) y $w_{ii} = 1$ (un nodo elegido se cubre a sí mismo); los pares ausentes del grafo tienen $w_{ij} = 0$.

Ganancia marginal. Para $S \subseteq V$ e $i \notin S$, sea $R_j(S) = \prod_{l \in S}(1 - w_{lj}) \in [0,1]$. Entonces

$$\Delta(i \mid S) := f(S \cup \{i\}) - f(S) = \sum_{j=1}^{M} a_j \big[R_j(S) - R_j(S)(1 - w_{ij})\big] = \sum_{j=1}^{M} a_j \, w_{ij} \, R_j(S). \qquad (1)$$

Cuidado: la fórmula (1) solo vale para $i \notin S$. Si $i \in S$, $\Delta(i \mid S) = 0$ pero la expresión de la derecha da $\sum_j a_j w_{ij}(1 - w_{ij}) \prod_{l \in S \setminus \{i\}}(1 - w_{lj})$, que en general no es cero. El código debe excluir los ya elegidos.

Lema 1.5 (propiedades de f; demostrado por nosotros)**.** *Si $a_j \geq 0$ y $w_{ij} \in [0,1]$ para todo i, j, entonces: (i) $f(\emptyset) = 0$; (ii) f es monótona: $S \subseteq T \Rightarrow f(S) \leq f(T)$; (iii) f es submodular: para $S \subseteq T \subseteq V$ e $i \in V \setminus T$, $\Delta(i \mid S) \geq \Delta(i \mid T)$.*

Demostración. (1) *(i)*: el producto vacío vale 1, luego cada término es $a_j(1-1) = 0$. (2) *(ii)*: por (1), $\Delta(i \mid S)$ es suma de productos de factores no negativos ($a_j \geq 0$, $w_{ij} \geq 0$, $R_j(S) \geq 0$ porque cada $1 - w_{lj} \in [0,1]$); luego $\Delta(i \mid S) \geq 0$. Si $S \subseteq T$, se agregan los elementos de $T \setminus S$ uno a uno y cada paso no disminuye f. (3) *(iii)*: si $S \subseteq T$, $R_j(T) = R_j(S) \cdot \prod_{l \in T \setminus S}(1 - w_{lj}) \leq R_j(S)$, porque el producto adicional está en $[0,1]$ y $R_j(S) \geq 0$. Multiplicando por $a_j w_{ij} \geq 0$ y sumando sobre j, usando (1) (válida porque $i \notin T \supseteq S$): $\Delta(i \mid T) \leq \Delta(i \mid S)$. ■

Observación 1.6. Las hipótesis $a_j \geq 0$ y $w_{ij} \in [0,1]$ son necesarias: si $w_{ij} > 1$, el factor $1 - w_{ij}$ es negativo y R_j puede cambiar de signo; si $a_j < 0$, f puede decrecer. El programa valida ambas al cargar el JSON.

Definición 1.7 (Problema Selección)**.**

- **Entrada.** N candidatos, M objetivos (en el JSON, $M = N$), pesos $a \in \mathbb{R}^M_{\geq 0}$, matriz $W \in [0,1]^{N \times M}$, cupo $k \in \mathbb{N}$ con $1 \leq k < N$, costos $c_i \in \mathbb{N}$ y presupuesto de costos $L \in \mathbb{N}$.
- **Salida.** Un subconjunto $S \subseteq V$ (los participantes de la ronda).
- **Restricciones** según el modo (D-03): modo `k`: $|S| \leq k$; modo `costo`: $\sum_{i \in S} c_i \leq L$; modo `ambos` (el de la solución base): $|S| \leq k$ **y** $\sum_{i \in S} c_i \leq L$.
- **Objetivo.** Maximizar $f(S)$.

Observación 1.8 ("tamaño k" frente a "a lo sumo k"). El enunciado pide "el subconjunto de tamaño k". En el modo `k` ambas lecturas tienen el mismo valor óptimo: si $|S| < k < N$, se completa S con nodos cualesquiera hasta tamaño k y, por monotonía (Lema 1.5), f no baja. En los modos con costo *no* son equivalentes (completar puede violar $\sum c_i \leq L$), por eso se usa "a lo sumo k".

Tamaño de la entrada: (N, k), y además M y L. Crecen: N (candidatos), k (cuántos se eligen; determina el número de pasos del voraz y el número $\binom{N}{k}$ de subconjuntos de la fuerza bruta), M (objetivos; determina el costo τ de evaluar una ganancia marginal) y, en los modos con costo, L. En el JSON $M = N$, así que el tamaño relevante es (N, k) (más L en los modos `costo`/`ambos`). La matriz W ocupa $N \cdot M$ números, de modo que leer la entrada ya cuesta $\Theta(NM)$.

Costo de una evaluación (τ). El enunciado expresa el voraz como $O(N \cdot k \cdot \tau)$, con τ el costo de evaluar una ganancia marginal. Manteniendo el vector $R_j = R_j(S)$ (inicial $R_j = 1$; tras elegir i, $R_j \leftarrow R_j(1 - w_{ij})$ en $O(M)$), la fórmula (1) da $\tau = \Theta(M)$, y el voraz cuesta $\Theta(k \cdot N \cdot M)$, es decir $\Theta(kN^2)$ con $M = N$ (deducción en la Sección 3.2). Evaluar $f(S)$ desde cero costaría $O(|S| \cdot M)$ y llevaría a $O(Nk^2M)$; por eso se usa la versión incremental.

Supuestos específicos. Grafo no dirigido y simétrico (D-10; la dirección no está especificada, Q-04); independencia de las influencias (modelo de cobertura probabilística, D-02); $q = 0$ para el caso de la profesora (D-07); desempates por menor índice (D-12). En una ronda t los candidatos son los nodos que no fallaron (con $q = 0$, todos).

Alcance. Esta parte es NP-difícil (Sección 2); se resuelve con el voraz de mayor ganancia marginal, con garantía $1 - 1/e$ *solo en el modo* `k` (Sección 3.4), y con fuerza bruta en instancias pequeñas para medir la razón real. Para el modo `ambos` se usa el mejor de tres voraces factibles (por ganancia, por razón ganancia/costo y el mejor nodo individual) y **no se afirma garantía teórica** para la combinación cupo+costo (R-01); la calidad se mide contra el exacto por fuerza bruta (con $N = 15$, $k = 5$: a lo sumo $\sum_{r=1}^{5} \binom{15}{r} = 4\,943$ subconjuntos no vacíos) y se reporta en la Sección 4 (en el caso de la profesora, $f(S)/f(S^*) = 1.0000$ en el modo `ambos`, con 3 893 subconjuntos factibles evaluados; origen: `resultado.json`, campo `comparacion_exacto`).

1.5. Supuestos de modelado y alcance de la entrega

ID	Supuesto	Dónde se usa
D-01	$u_i(x) = a_i \ln(1+x)$, $x \in \mathbb{N}$, $a_i =$ `coeficiente_utilidad`	Parte 1, Lema 1.1
D-02	$f(S) = \sum_j a_j[1 - \prod_{i \in S}(1 - w_{ij})]$, $w_{ii} = 1$; familia general $N \times M$	Parte 2, Lema 1.5, reducción
D-03	Base con cupo **y** costo a la vez ($\lvert S\rvert \le k$, $\sum c_i \le L$); modos `k` y `costo` por bandera	Def. 1.7
D-04	Alcance: primera entrega y extras si alcanza (P0→P1→P2)	Toda la entrega
D-05	Python $\ge$ 3.10; numpy, scipy, matplotlib; pytest + hypothesis	Implementación
D-06	Topes por nodo $r_i = R_i -$ gastado en rondas previas	Def. 1.3
D-07	$q = 0$ en el caso de la profesora; barrido de q como extra	Rondas
D-08	Partes resueltas en cadena por ronda (Parte 2 → Parte 1)	Contexto
D-10	Grafo no dirigido y simétrico	W
D-11	Tiempos con `perf_counter`, mediana de ≥ 5 repeticiones, semilla 20261105	Validación
D-12	Desempates por menor índice de nodo	Ambos voraces
D-13	ln natural; B y costos enteros; `float64` con tolerancia 10^{-9} en pruebas	Ambas partes

Cuadro 1: Supuestos de modelado (D-09 fija el formato del informe; no afecta al modelo).

Qué cubre esta primera entrega. Formalización de ambas partes, prueba de NP-dureza de la Parte 2, optimalidad del voraz de la Parte 1 (Teorema 3.1), aplicación de la cota $1 - 1/e$ a nuestra f (Teorema 3.12) y ejecución de la solución base sobre el caso de la profesora (Sección 4).

Qué queda para entregas posteriores. Deducción completa de $T(n)$ con todas las constantes y validación experimental con ≥ 5 tamaños (P1); variantes obligatorias: costos con la versión de Khuller et al. [10], T rondas con fallas y políticas de gasto, y nodos relacionados (P2).

2. NP-dureza de la selección de participantes

Plan. (1) Se fija el problema de origen, Cobertura Máxima, y se justifica que es NP-completo. (2) Se define la versión de decisión de nuestra Parte 2 (Sel-Dec) y se prueba que está en NP. (3) Se construye una transformación polinomial φ de instancias de Cobertura Máxima a instancias de Sel-Dec y se calcula su costo. (4) Se prueba la equivalencia de respuestas en *ambos* sentidos. (5) Se trabaja un ejemplo a mano. (6) Se refuerza el resultado para la subfamilia que realmente aparece en el JSON. (7) Se concluye qué componente es NP-difícil.

2.1. Problema de origen: Cobertura Máxima

Definición 2.1 (CobMax-Dec)**. Instancia:** un universo finito $U = [m]$, una colección $S_1, \ldots, S_n$ de subconjuntos de U y enteros $k, t \geq 0$. **Pregunta:** ¿existe $I \subseteq [n]$ con $|I| \leq k$ y $|\bigcup_{i \in I} S_i| \geq t$?

Definición 2.2 (CobConj-Dec, Cobertura de Conjuntos)**. Instancia:** $U = [m]$, $S_1, \ldots, S_n \subseteq U$, entero k. **Pregunta:** ¿existe $I \subseteq [n]$, $|I| \leq k$, con $\bigcup_{i \in I} S_i = U$?

Lema 2.3 (CobMax-Dec es NP-completo)**.**

Demostración. (1) *Pertenece a NP*: el certificado es I; se verifica $|I| \leq k$ y se calcula la unión con un arreglo de m banderas en $O(m + \sum_{i \in I} |S_i|) \subseteq O(nm)$. (2) *Es NP-difícil*: CobConj-Dec es NP-completo; es uno de los 21 problemas de Karp [5] (*Set Covering*) y el enunciado del curso lo usa como conocido (Proposición 4.3.1 de [1]). Dada una instancia $(U, \{S_i\}, k)$ de CobConj-Dec, se produce $(U, \{S_i\}, k, t = m)$ de CobMax-Dec en tiempo lineal en el tamaño de la entrada. Como toda unión está contenida en U, $|\bigcup_{i \in I} S_i| \geq m \iff \bigcup_{i \in I} S_i = U$; por tanto las respuestas coinciden para cada I y, en particular, una instancia es positiva si y solo si la otra lo es. ■

2.2. Versión de decisión de la Parte 2 y pertenencia a NP

Definición 2.4 (Sel-Dec)**. Instancia:** enteros $N \geq 2$, $M \geq 1$; pesos racionales $a_j \geq 0$ ($j \in [M]$); matriz racional $W \in ([0,1] \cap \mathbb{Q})^{N \times M}$; enteros k con $1 \leq k < N$; umbral racional t. **Pregunta:** ¿existe $A \subseteq [N]$ con $|A| \leq k$ y $f(A) \geq t$, donde $f(A) = \sum_j a_j [1 - \prod_{i \in A}(1 - w_{ij})]$?

Esta es la versión de decisión del modo `k` de la Definición 1.7. Se usan datos racionales para que la entrada tenga longitud finita en bits; el JSON trae decimales finitos, que son racionales.

Proposición 2.5 (Sel-Dec $\in$ NP; demostrado por nosotros)**.**

Demostración. (1) *Certificado*: el conjunto A, de tamaño $\leq k < N$, codificado con $O(N \log N)$ bits. (2) *Verificador*: comprueba $|A| \leq k$ en $O(N)$ y calcula $f(A)$ con aritmética racional exacta: para cada j, el producto de $|A|$ factores $(1 - w_{ij})$; en total $O(|A| \cdot M) \subseteq O(NM)$ operaciones aritméticas; luego compara con t. (3) *Polinomialidad en bits*: el producto de $|A|$ racionales tiene numerador y denominador con a lo sumo la suma de las longitudes de los factores, es decir, $O(|A| \cdot \ell)$ bits si cada w_{ij} ocupa ℓ bits; cada suma y producto de racionales de esa longitud cuesta tiempo polinomial en ella. Por tanto el verificador es polinomial en la longitud de la instancia. (4) A es aceptado si y solo si es testigo de una respuesta positiva, así que Sel-Dec $\in$ NP. ■

2.3. La transformación φ y su costo

Sea $(U = [m], S_1, \ldots, S_n, k, t)$ una instancia de CobMax-Dec, con cada S_i dado como lista de elementos de $[m]$.

Caso borde ($k \geq n$). Sel-Dec exige $k < N$ (Definición 3.3.1 del enunciado: $k < N$). Si $k \geq n$, la instancia de origen se decide directamente: es positiva si y solo si $|\bigcup_{i=1}^{n} S_i| \geq t$ (tomar todos los conjuntos es lo mejor, porque la unión es monótona), lo cual se calcula en $O(nm)$. Según la respuesta, φ devuelve una instancia fija de Sel-Dec: positiva Y_0 ($N = 2$, $M = 1$, $a_1 = 1$, $w_{11} = w_{21} = 0$, $k = 1$, $t = 0$; la respuesta es sí con $A = \emptyset$, pues $f(\emptyset) = 0 \geq 0$) o negativa N_0 (los mismos datos con $t = 1$; como todos los w son 0, $f \equiv 0 < 1$).

Caso general ($k < n$). φ produce:

- **candidatos**: un nodo i por conjunto S_i, $N := n$;
- **objetivos**: uno por elemento del universo, $M := m$, con $a_j := 1$;
- **influencias**: $w_{ij} := 1$ si $j \in S_i$ y $w_{ij} := 0$ si $j \notin S_i$;
- **mismo cupo y mismo umbral**: k y t; sin costos (modo `k`).

Algoritmo 1 Transformación φ (caso $k < n$)

Forma/Tipo/Nombre: E: entero n, entero m, listas $S_1..S_n$ de enteros en $[m]$, enteros k, t; S: entero N, entero M, arreglo real $a[1..M]$, matriz real $w[1..N][1..M]$, enteros k, t.
Precondición: $k < n$, cada $S_i \subseteq [m]$.
Poscondición: (N, M, a, w, k, t) es la instancia de Sel-Dec descrita arriba.

1: **para** $i = 1$ **hasta** n **hacer**
2: **para** $j = 1$ **hasta** m **hacer**
3: $w[i][j] \leftarrow 0$
4: **para cada** $e \in S_i$ **hacer**
5: $w[i][e] \leftarrow 1$
6: **para** $j = 1$ **hasta** m **hacer**
7: $a[j] \leftarrow 1$
8: **devolver** (n, m, a, w, k, t)

Línea	Instrucción	Ejecuciones	Subtotal
1	`para` i (cabeza)	$n+1$	$c_1(n+1)$
2	`para` j (cabeza)	$n(m+1)$	$c_2\, n(m+1)$
3	$w[i][j] \leftarrow 0$	nm	$c_3\, nm$
4	`para cada` $e \in S_i$ (cabeza)	$\sum_i(\lvert S_i\rvert + 1) \le n(m+1)$	$\le c_4\, n(m+1)$
5	$w[i][e] \leftarrow 1$	$\sum_i \lvert S_i\rvert \le nm$	$\le c_5\, nm$
6	`para` j (cabeza)	$m+1$	$c_6(m+1)$
7	$a[j] \leftarrow 1$	m	$c_7\, m$
8	devolver	1	c_8

Cuadro 2: Costeo de φ (cabeza de ciclo = cuerpo +1). Se usó $\sum_i |S_i| \le nm$ porque cada $S_i \subseteq [m]$.

Sumando de adentro hacia afuera: $T_\varphi(n, m) \le \alpha\, nm + \beta\, n + \gamma\, m + \delta$ para constantes $\alpha, \beta, \gamma, \delta > 0$, y como la línea 3 se ejecuta exactamente nm veces, $T_\varphi(n, m) \in \Theta(n \cdot m) = \Theta(N \cdot |U|)$. Con el caso borde ($O(nm)$) el costo total sigue en $O(N \cdot |U|)$: **la transformación es polinomial**. Además la instancia producida tiene tamaño $\Theta(nm)$ números de un bit más k y t, polinomial en el tamaño de la de origen.

2.4. Equivalencia de respuestas

Lema 2.6 (la utilidad construida es la cobertura; demostrado por nosotros)**.** *En la instancia $\varphi(U, \{S_i\}, k, t)$ con $k < n$, para todo $A \subseteq [n]$: $f(A) = \left|\bigcup_{i \in A} S_i\right|$.*

Demostración. (1) Fijo $j \in [m]$. Cada factor $1 - w_{ij}$ vale 0 si $j \in S_i$ y 1 si $j \notin S_i$ (definición de w). (2) Un producto de factores en $\{0, 1\}$ vale 0 si y solo si algún factor es 0. Luego $\prod_{i \in A}(1 - w_{ij}) = 0 \iff \exists i \in A : j \in S_i \iff j \in \bigcup_{i \in A} S_i$ (si $A = \emptyset$ el producto vacío es 1 y la unión vacía no contiene a j: también coincide). (3) Por (2), $1 - \prod_{i \in A}(1 - w_{ij}) = \mathbf{1}[j \in \bigcup_{i \in A} S_i]$. (4) Con $a_j = 1$: $f(A) = \sum_{j=1}^{m} \mathbf{1}[j \in \bigcup_{i \in A} S_i] = \left|\bigcup_{i \in A} S_i\right|$. ■

Teorema 2.7 (equivalencia; demostrado por nosotros)**.** *Para toda instancia* $X = (U, \{S_i\}_{i=1}^n, k, t)$ *de* CobMax-Dec*:* X *es positiva* $\iff$ $\varphi(X)$ *es positiva para* Sel-Dec*.*

Demostración. Caso $k \geq n$*.* Por construcción, $\varphi(X) = Y_0$ (positiva) exactamente cuando $|\bigcup_{i=1}^n S_i| \geq t$, que es exactamente cuando X es positiva (tomar $I = [n]$, $|I| = n \leq k$, maximiza la unión). En otro caso $\varphi(X) = N_0$ (negativa) y X es negativa.

Caso $k < n$*.* Se identifica el candidato i con el conjunto S_i.

($\Rightarrow$) Supongamos X positiva: existe $I \subseteq [n]$ con $|I| \leq k$ y $|\bigcup_{i \in I} S_i| \geq t$.

(1) Sea $A := I \subseteq [n] = [N]$. Entonces $|A| = |I| \leq k$.

(2) Por el Lema 2.6, $f(A) = |\bigcup_{i \in I} S_i| \geq t$.

(3) Luego A es testigo de que $\varphi(X)$ es positiva.

($\Leftarrow$) Supongamos $\varphi(X)$ positiva: existe $A \subseteq [N] = [n]$ con $|A| \leq k$ y $f(A) \geq t$.

(1) Sea $I := A$. Entonces $|I| \leq k$.

(2) Por el Lema 2.6, $|\bigcup_{i \in I} S_i| = f(A) \geq t$.

(3) Luego I es testigo de que X es positiva.

Los dos sentidos usan la misma correspondencia $I \leftrightarrow A$ y el mismo Lema 2.6; no hay saltos: el valor de la función objetivo es *idéntico* en ambos lados para cada subconjunto. ■

Corolario 2.8 (demostrado por nosotros a partir del Lema 2.3)**.** *Sel-Dec es NP-completo; en particular, es NP-difícil.*

Demostración. Por el Lema 2.3, CobMax-Dec es NP-difícil. φ es computable en tiempo $O(N \cdot |U|)$ (Tabla 2) y preserva respuestas (Teorema 2.7): CobMax-Dec $\leq_p$ Sel-Dec. Una reducción polinomial desde un problema NP-difícil hace NP-difícil al destino. Con la Proposición 2.5, Sel-Dec es NP-completo. ■

2.5. Ejemplo trabajado a mano

Ejemplo 2.9. $U = \{1,2,3,4,5\}$, $S_1 = \{1,2,3\}$, $S_2 = \{3,4\}$, $S_3 = \{4,5\}$, $S_4 = \{1,5\}$, $k = 2$, $t = 5$. Como $k = 2 < n = 4$, se aplica el caso general: $N = 4$, $M = 5$, $a = (1,1,1,1,1)$ y

$$W = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \end{pmatrix} \quad \text{(fila } i = S_i,\ \text{columna } j = \text{elemento)}.$$

Cálculo de f con la fórmula para $A = \{1,3\}$: los productos por columna $\prod_{i \in A}(1 - w_{ij})$ son j=1: $(1-1)(1-0) = 0$; j=2: $(0)(1) = 0$; j=3: $(0)(1) = 0$; j=4: $(1)(0) = 0$; j=5: $(1)(0) = 0$. Luego $f(\{1,3\}) = 5 \cdot (1-0) = 5 \geq t$. Para $A = \{1,2\}$: la columna 5 da $(1-0)(1-0) = 1$ y las demás 0, así que $f(\{1,2\}) = 4$.

A	$\bigcup_{i \in A} S_i$	$\lvert\bigcup\rvert$	$f(A)$ por la fórmula
$\{1,2\}$	$\{1,2,3,4\}$	4	4
$\{1,3\}$	$\{1,2,3,4,5\}$	5	5
$\{1,4\}$	$\{1,2,3,5\}$	4	4
$\{2,3\}$	$\{3,4,5\}$	3	3
$\{2,4\}$	$\{1,3,4,5\}$	4	4
$\{3,4\}$	$\{1,4,5\}$	3	3

($\Rightarrow$) $I = \{1, 3\}$ cubre U ($|\bigcup| = 5 \geq 5$) y $A = \{1, 3\}$ da $f(A) = 5 \geq 5$. ($\Leftarrow$) El único A con $|A| \leq 2$ y $f(A) \geq 5$ es $\{1, 3\}$ (los de tamaño 1 valen $|S_i| \leq 3$), y corresponde a $I = \{1, 3\}$. Ambas instancias son positivas.

Instancia negativa: mismos conjuntos, $k = 1$, $t = 4$. Ningún conjunto tiene 4 elementos ($|S_i| \leq 3$) y, en φ, $f(\{i\}) = |S_i| \leq 3 < 4$. Ambas son negativas.

2.6. Refuerzo: la dureza se mantiene en la subfamilia del JSON

La instancia de la profesora tiene una estructura particular: candidatos y objetivos son los mismos nodos ($M = N$), W es simétrica, $w_{ii} = 1$ y $a_j > 0$. La reducción anterior produce W rectangular y no simétrica, así que alguien podría objetar que la subfamilia que de verdad resolvemos podría ser fácil. No lo es:

Proposición 2.10 (demostrado por nosotros)**.** *Sel-Dec sigue siendo NP-completo restringido a instancias con $M = N$, W simétrica con $w_{ij} \in \{0, 1\}$, $w_{ii} = 1$ y $a_j > 0$.*

Demostración. Sea $X = (U, \{S_i\}_{i=1}^n, k, t)$ de CobMax-Dec con $k < n$ (el caso $k \geq n$ se trata como antes, con instancias fijas que también cumplen la restricción: p. ej. $N = 2$, $W = I_2$, $a = (1, 1)$, $k = 1$, $t = 0$ o $t = 3$).

(1) *Limpieza.* Sea $U' = \bigcup_{i=1}^n S_i$, $m' = |U'|$. Los elementos de $U \setminus U'$ no los cubre ningún conjunto, así que $|\bigcup_{i \in I} S_i|$ no cambia al quitarlos: X y $X' = (U', \{S_i\}, k, t)$ tienen la misma respuesta. Costo $O(nm)$.

(2) *Construcción.* Grafo bipartito con vértices $s_1, \ldots, s_n$ (conjuntos) y $e_1, \ldots, e_{m'}$ (elementos); $N = M = n + m'$. $w = 1$ en la diagonal; $w_{s_i e_j} = w_{e_j s_i} = 1$ si $j \in S_i$; 0 en otro caso. W es simétrica con diagonal 1. Pesos $a(e_j) = 1$ y $a(s_i) = \varepsilon := 1/(n + 1) > 0$. Mismos k y t; $k < n < N$. Costo $O((n + m)^2)$, polinomial.

(3) *Descomposición de f.* Para $A \subseteq$ vértices, un vértice v queda cubierto ($1 - \prod_{u \in A}(1 - w_{uv}) = 1$) si y solo si $v \in A$ o algún vecino de v está en A (mismo argumento del Lema 2.6). Sean $C(A)$ el número de e_j cubiertos y $D(A)$ el de s_i cubiertos. Entonces $f(A) = C(A) + \varepsilon D(A)$ con $0 \leq \varepsilon D(A) \leq n/(n + 1) < 1$.

(4) ($\Rightarrow$) Si I, $|I| \leq k$, cumple $|\bigcup_{i \in I} S_i| \geq t$, sea $A = \{s_i : i \in I\}$. Cada e_j con $j \in \bigcup_{i \in I} S_i$ es vecino de algún $s_i \in A$, luego $C(A) \geq t$ y $f(A) \geq C(A) \geq t$.

(5) ($\Leftarrow$) Si $|A| \leq k$ y $f(A) \geq t$: por (3), $C(A) > t - 1$, y como $C(A)$ y t son enteros, $C(A) \geq t$. Se construye I: por cada $s_i \in A$ se agrega i; por cada $e_j \in A$ se agrega un índice $i(j)$ con $j \in S_{i(j)}$ (existe por la limpieza (1)). Así $|I| \leq |A| \leq k$. Todo e_j cubierto por A está en $\bigcup_{i \in I} S_i$: o bien es vecino de un $s_i \in A$ (y $j \in S_i$, $i \in I$), o bien $e_j \in A$ (y $j \in S_{i(j)}$, $i(j) \in I$). Luego $|\bigcup_{i \in I} S_i| \geq C(A) \geq t$.

(6) La pertenencia a NP es la Proposición 2.5. Por (4)–(5) y el Lema 2.3, la subfamilia es NP-completa.

■

Observación 2.11. En la subfamilia los pesos de W son 0/1, mientras que el JSON trae pesos en el intervalo $(0, 1)$ y un grafo completo. La Proposición 2.10 prueba la dureza para el *esquema* (que admite cualquier grafo y cualquier peso en $[0, 1]$), no para la instancia concreta de 15 nodos, que es finita y se resuelve exactamente por fuerza bruta.

2.7. Consecuencias y conclusión

Corolario 2.12 (demostrado por nosotros)**.** *(a) La versión de optimización de la Parte 2 (modo* `k`*) es NP-difícil: si se resolviera en tiempo polinomial,* Sel-Dec *se decidiría comparando $f(S^*)$ con t, y entonces P=NP. (b) Los modos* `costo` *y* `ambos` *también son NP-difíciles: toda instancia del modo* `k` *es una instancia del modo* `costo` *con $c_i = 1$ y $L = k$ (pues $\sum_{i \in S} 1 \le k \iff |S| \le k$) y del modo* `ambos` *con $c_i = 0$, $L = 0$. (c)* Sel-Dec *es NP-difícil* en sentido fuerte*: en las instancias producidas por φ todos los números (a_j, w_{ij}, k, t) están acotados por* $\max(n, m)$*, polinomial en la longitud. Por tanto no existe un algoritmo pseudo-polinomial para la Parte 2 salvo que P=NP (definición de dureza fuerte según Garey y Johnson [6]* ***[VERIFICAR]*** *para la página exacta).*

Conclusión.

- **Componente NP-difícil: la Parte 2 (selección de participantes)**, en sus tres modos, incluida la subfamilia del esquema JSON (simétrica, $w_{ii} = 1$, $a > 0$). Por eso no se busca un algoritmo exacto polinomial para ella: se usa un voraz con garantía $1 - 1/e$ en el modo `k` (Sección 3.4) y fuerza bruta solo para instancias pequeñas.
- **Parte 1 (reparto)**: tiene un algoritmo exacto en $O(N + B \log N)$ (Sección 3.1), polinomial en N y *pseudo-polinomial* en B. No afirmamos aquí nada sobre su dureza en la longitud en bits de B; solo que el algoritmo implementado es exacto y pseudo-polinomial. No hay contradicción con la NP-dureza: son problemas distintos, y la dificultad del proyecto está en *a quién elegir*, no en *cuánto dar*.

3. Algoritmos base: optimalidad, costo y garantía

3.1. Parte 1: el voraz por ganancia marginal es óptimo

Algoritmo 2 Reparto (voraz por ganancia marginal con montículo y topes)

Forma/Tipo/Nombre: E: entero N; E: arreglo real $a[1..N]$; E: arreglo entero $r[1..N]$; E: entero B; S: arreglo entero $x[1..N]$.

Precondición: $N \ge 1$, $a_i > 0$, $r_i \in \mathbb{N}$, $B \in \mathbb{N}$.

Poscondición: $\sum_i x_i \le B$, $0 \le x_i \le r_i$ y $U(x) = \sum_i a_i \ln(1 + x_i)$ es máxima entre los repartos factibles.

```
1:  para i = 1 hasta N hacer
2:      x[i] ← 0
3:  H ← arreglo vacío
4:  para i = 1 hasta N hacer
5:      si r[i] > 0 entonces
6:          agregar el par (g_i(0), i) al final de H
7:  ConstruirMontículoMáx(H)              ▷ orden: mayor g; empate, menor i (D-12)
8:  b ← 0
9:  mientras b < B y H ≠ ∅ hacer
10:     (g, i) ← ExtraerMáx(H)
11:     x[i] ← x[i] + 1; b ← b + 1
12:     si x[i] < r[i] entonces
13:         Insertar(H, (g_i(x[i]), i))
14: devolver x
```

Notación: $x^{(j)}$ es el vector x tras j iteraciones del ciclo de la línea 9; $U(y) = \sum_i u_i(y_i)$; e_p es el vector unitario del nodo p. Como $u_i(0) = 0$, $U(y) = \sum_i \sum_{s=0}^{y_i - 1} g_i(s)$.

Teorema 3.1 (optimalidad del voraz con topes; demostrado por nosotros)**.** *Si cada u_i cumple $u_i(0) = 0$ y su ganancia marginal g_i es positiva y no creciente (por el Lema 1.1, esto vale para $u_i = a_i \ln(1 + x)$ con $a_i > 0$), el Algoritmo 2 termina tras $J = \min(B, \sum_i r_i)$ iteraciones y devuelve un reparto óptimo de la Definición 1.3.*

Demostración. Se demuestra con un **invariante de lazo** para el ciclo de la línea 9 y un **argumento de intercambio** en el mantenimiento. Existe al menos un óptimo porque el conjunto de repartos factibles es finito ($0 \le y_i \le r_i$) y no vacío ($y = 0$).

Invariante $\mathcal{I}(j)$, antes de la iteración $j + 1$: (a) $x^{(j)}$ es factible y $b = \sum_i x_i^{(j)} = j$; (b) H contiene exactamente los nodos i con $x_i^{(j)} < r_i$, cada uno una sola vez y con clave $g_i(x_i^{(j)})$; (c) existe un reparto óptimo y con $y \ge x^{(j)}$ componente a componente.

Inicialización ($j = 0$). (a) $x^{(0)} = 0$, factible, $b = 0$. (b) Las líneas 4–6 insertan exactamente los i con $r_i > 0 = x_i^{(0)}$, con clave $g_i(0)$, y la línea 7 solo reordena. (c) Todo óptimo y cumple $y \ge 0 = x^{(0)}$.

Mantenimiento. Supongamos $\mathcal{I}(j - 1)$ y que se ejecuta la iteración j (luego $j \le B$ y $H \neq \emptyset$). Abreviamos $x = x^{(j-1)}$. Sea p el nodo extraído en la línea 10.

(1) Por (b), p cumple $x_p < r_p$ y $g_p(x_p) = \max\{g_i(x_i) : x_i < r_i\}$ (propiedad de ExtraerMáx). Tras las líneas 11–13, $x^{(j)} = x + e_p$, que es factible porque $x_p + 1 \le r_p$ y $\sum_i x_i^{(j)} = j \le B$: se cumple (a). Si $x_p + 1 < r_p$, p se reinserta con clave $g_p(x_p + 1)$; si no, queda fuera; los demás nodos no cambian: se cumple (b).

(2) Para (c), sea y el óptimo dado por $\mathcal{I}(j - 1)$, con $y \ge x$. *Caso A*: $y_p > x_p$. Entonces $y \ge x + e_p = x^{(j)}$ y el mismo y sirve.

(3) *Caso B*: $y_p = x_p$. Primero, $\sum_i y_i = B$: si fuera $\sum_i y_i < B$, el vector $y + e_p$ sería factible ($y_p + 1 = x_p + 1 \le r_p$) y $U(y + e_p) - U(y) = g_p(y_p) > 0$ (hipótesis $g_p > 0$), lo que contradice la optimalidad de y.

(4) Como $\sum_i y_i = B \ge j > j - 1 = \sum_i x_i$ e $y \ge x$, existe q con $y_q > x_q$; y $q \neq p$ porque $y_p = x_p$.

(5) *q era candidato en la iteración j*: $x_q < y_q \le r_q$, así que $q \in H$ por (b), y por (1) $g_q(x_q) \le g_p(x_p)$.

(6) Sea $y' = y - e_q + e_p$. Es factible: $\sum_i y'_i = \sum_i y_i \le B$; $y'_p = x_p + 1 \le r_p$; $y'_q = y_q - 1 \ge x_q \ge 0$.

(7) $U(y') - U(y) = g_p(y_p) - g_q(y_q - 1) = g_p(x_p) - g_q(y_q - 1)$. Como $y_q - 1 \ge x_q$ y g_q es no creciente (concavidad), $g_q(y_q - 1) \le g_q(x_q) \le g_p(x_p)$ por (5). Luego $U(y') \ge U(y)$, y como y es óptimo, y' también lo es.

(8) $y' \ge x^{(j)}$: $y'_p = x_p + 1 = x_p^{(j)}$, $y'_q = y_q - 1 \ge x_q = x_q^{(j)}$ y las demás componentes no cambian. Se cumple (c).

Terminación. Cada iteración aumenta b en 1 y disminuye en 1 la cantidad $\sum_i (r_i - x_i) \ge 0$, así que el ciclo termina; por (a) y (b), sale cuando $b = B$ o cuando $H = \emptyset$, es decir, cuando $x_i = r_i$ para todo i; en ambos casos $J = \min(B, \sum_i r_i)$. Sea $y \ge x^{(J)}$ el óptimo dado por (c). Si $b = B$: $\sum_i y_i \le B = \sum_i x_i^{(J)}$ e $y \ge x^{(J)}$ obligan a $y = x^{(J)}$. Si $H = \emptyset$: $x_i^{(J)} = r_i \ge y_i \ge x_i^{(J)}$, luego $y = x^{(J)}$. En ambos casos $x^{(J)}$ es óptimo. ∎

Observación 3.2 (dónde se usa cada hipótesis). **Concavidad** (g no creciente): en el paso (7). $g > 0$: solo en el paso (3); si $g \ge 0$ (p. ej. $a_i = 0$), $y + e_p$ también es óptimo y cumple $y + e_p \ge x^{(j)}$, así que el teorema sigue valiendo. **Topes**: el paso (5) es el único punto delicado que añaden: hay que comprobar que q todavía estaba en el montículo, y eso lo garantiza $y_q \le r_q$. **Desempates**: no se usan; cualquier regla de desempate da un óptimo (D-12 solo fija cuál).

Contraejemplo 3.3 (sin concavidad el voraz falla; un voraz se demuestra óptimo o se refuta con un contraejemplo [3, cap. 5])**.** Dos nodos, $B = 2$, sin topes, con ganancias marginales $g_1 = (1, 10)$ (convexa: la segunda unidad vale más) y $g_2 = (2, 1)$. Voraz: paso 1, máx$\{g_1(0), g_2(0)\} =$ máx$\{1, 2\}$, elige el nodo 2; paso 2, máx$\{g_1(0), g_2(1)\} =$ máx$\{1, 1\}$, empate, elige el nodo 1 (menor índice). Utilidad $2 + 1 = 3$. Óptimo: $x = (2, 0)$ con utilidad $1 + 10 = 11$. El paso (7) falla porque $g_1(1) > g_1(0)$.

Observación 3.4 (prueba alternativa por "las B mayores ganancias")*.* $U(x) = \sum_i \sum_{s<x_i} g_i(s)$: todo reparto factible toma, de cada nodo, un *prefijo* de su lista $g_i(0) \geq g_i(1) \geq \cdots \geq g_i(r_i - 1)$, y en total a lo sumo B elementos del multiconjunto $G = \{g_i(s) : i \in S,\ 0 \leq s < r_i\}$. Como todos son positivos, $U(x) \leq$ suma de los mín$(B, |G|)$ mayores de G. Como cada lista está ordenada de mayor a menor, el máximo de G entre lo no tomado es siempre la cabeza de alguna lista, que es lo que extrae el voraz; así toma exactamente los mín$(B, |G|)$ mayores y alcanza la cota.

3.1.1. Costo del Algoritmo 2

Sea $N' = |\{i : r_i > 0\}| \leq N$ y $J = \text{mín}(B, \sum_i r_i) \leq B$. Por el invariante (b), cada nodo está a lo sumo una vez en H, así que $|H| \leq N' \leq N$ siempre. Costos del montículo binario (Notas de clase §3.1 [2]; [4, cap. 6]): construir con HEAPIFY de abajo hacia arriba cuesta $O(|H|)$; EXTRAERMÁX e INSERTAR cuestan $O(1+\log|H|)$. Se asume costo unitario para las operaciones aritméticas reales, incluido ln (D-13).

Línea	Instrucción	Ejecuciones	Costo por ejecución
1	`para` i (cabeza)	$N+1$	c_1
2	$x[i] \leftarrow 0$	N	c_2
3	$H \leftarrow \emptyset$	1	c_3
4	`para` i (cabeza)	$N+1$	c_4
5	`si` $r[i] > 0$	N	c_5
6	agregar $(g_i(0), i)$	N'	c_6
7	CONSTRUIRMONTÍCULOMÁX	1	$\leq c_7 N'$
8	$b \leftarrow 0$	1	c_8
9	`mientras` (cabeza)	$J+1$	c_9
10	EXTRAERMÁX	J	$\leq c_{10}(1+\log N)$
11	incrementos	J	c_{11}
12	`si` $x[i] < r[i]$	J	c_{12}
13	INSERTAR	$\leq J$	$\leq c_{13}(1+\log N)$
14	devolver	1	c_{14}

Sumando: $T(N, B) \leq (c_1+c_2+c_4+c_5+c_6+c_7)N+(c_9+c_{11}+c_{12})J+(c_{10}+c_{13})J(1+\log N)+c$, y con $J \leq B$:

$$T(N, B) \leq \alpha N + \beta B(1 + \log N) + \gamma \implies T(N, B) \in O(N + B \log N).$$

Mejor caso ($B = 0$ o todos los $r_i = 0$): $J = 0$ y $T_{\text{mejor}} \in \Theta(N)$. **Peor caso**: $T_{\text{peor}}(N, B) \in \Theta(N + B(1 + \log N))$, que para $N \geq 2$ es $\Theta(N + B \log N)$. La cota superior es la de arriba. La inferior se alcanza con topes holgados ($r_i > B$ para todo i): entonces $J = B$, el montículo conserva sus N elementos durante todas las extracciones, y en la implementación de CPython cada extracción baja siempre hasta una hoja, sea cual sea el valor de las claves (Lema 3.6). Por eso cada extracción cuesta $\Omega(1 + \log N)$ y no solo $O(\log N)$. La deducción completa, con instancia testigo y caso promedio, está en la Proposición 3.7. **Espacio**: el pseudocódigo usa $\Theta(N)$ (arreglos x, a, r y H). El código guarda además la traza de las J unidades, así que usa $\Theta(N + J) \subseteq O(N + B)$.

Pseudo-polinomialidad. La entrada ocupa $\Theta(N\beta+\log B)$ bits, pero el número de iteraciones J puede ser $B = 2^{\log B}$: el algoritmo es polinomial en N y en el *valor* de B, pero exponencial en la longitud de B. Por ejemplo, duplicar la longitud en bits de B (de $\log B$ a $2\log B$) eleva B al cuadrado y con él el número de iteraciones.

Observación 3.5 (presupuesto continuo, *water-filling*; esbozo, la deducción completa es T11). Si $x_i \in [0, r_i]$ es real, el problema máx $\sum_i a_i \ln(1 + x_i)$, $\sum_i x_i \le B$, es cóncavo con restricciones lineales y sus condiciones de Karush–Kuhn–Tucker son suficientes. Igualando la derivada $a_i/(1+x_i)$ a un multiplicador común λ se obtiene $x_i(\lambda) = \min\{r_i, \max\{0,\, a_i/\lambda - 1\}\}$. Si $\sum_i r_i \le B$, la solución es $x = r$; si no, $\lambda > 0$ se elige con $\sum_i x_i(\lambda) = B$. La función $\lambda \mapsto \sum_i x_i(\lambda)$ es continua, no creciente y lineal a trozos en $1/\lambda$, con $2N$ puntos de quiebre ($\lambda = a_i$ y $\lambda = a_i/(1 + r_i)$): ordenarlos y recorrerlos cuesta $O(N \log N)$, **independiente de** B. Así el caso continuo elimina el factor pseudo-polinomial. El nombre viene de "llenar con agua" recipientes de fondo $1/a_i$ hasta el mismo nivel $1/\lambda$.

3.2. Parte 2: voraz de mayor ganancia marginal y su costo

Algoritmo 3 VorazSelección (modo `k`, con R_j incremental)

Forma/Tipo/Nombre: E: enteros N, M, k; E: arreglo real $a[1..M]$; E: matriz real $w[1..N][1..M]$; S: conjunto S.

Precondición: $1 \le k < N$, $a_j \ge 0$, $w_{ij} \in [0, 1]$.

Poscondición: $|S| = k$ y S es la salida del voraz de mayor ganancia marginal (desempate por menor índice).

```
 1: para j = 1 hasta M hacer
 2:     R[j] ← 1
 3: S ← ∅; enS[1..N] ← falso
 4: para t = 1 hasta k hacer
 5:     g_máx ← −∞; mejor ← 0
 6:     para i = 1 hasta N hacer
 7:         si no enS[i] entonces
 8:             d ← 0
 9:             para j = 1 hasta M hacer
10:                 d ← d + a[j] · w[i][j] · R[j]
11:             si d > g_máx entonces
12:                 g_máx ← d; mejor ← i
13:     S ← S ∪ {mejor}; enS[mejor] ← verdadero
14:     para j = 1 hasta M hacer
15:         R[j] ← R[j] · (1 − w[mejor][j])
16: devolver S
```

Correctitud (invariante). Por lazo invariante (inicialización, mantenimiento y terminación) [2, §2.3]. Antes de la iteración t del ciclo de la línea 4: $|S| = t - 1$ y $R[j] = \prod_{l \in S}(1 - w_{lj})$ para todo j. Inicialización: $S = \emptyset$, $R \equiv 1$ (producto vacío). Mantenimiento: las líneas 6–12 calculan, para cada $i \notin S$, $d = \sum_j a_j w_{ij} R_j = \Delta(i \mid S)$ por (1), y guardan el primero de mayor valor (">" estricto = menor índice); siempre hay candidato porque quedan $N - t + 1 \ge 2$ nodos fuera de S y $d \ge 0 > -\infty$; las líneas 13–15 agregan $mejor$ y multiplican R_j por $(1 - w_{mejor,j})$, lo que restablece el invariante para $S \cup \{mejor\}$. Terminación: tras k iteraciones, $|S| = k$.

Costo, de adentro hacia afuera. Costeo línea a línea, resolviendo las sumatorias de la más interna a la más externa [2, §2.1]. Sea P el número de veces que se ejecuta la línea 8 (candidatos

evaluados): en la iteración t hay $N-(t-1)$ nodos fuera de S, luego

$$P=\sum_{t=1}^{k}(N-t+1)=kN-\frac{k(k-1)}{2}.$$

Como $k\le N-1$, $\frac{k(k-1)}{2}<\frac{kN}{2}$, así que $\frac{kN}{2}<P\le kN$, es decir $P\in\Theta(kN)$.

Línea	Instrucción	Ejecuciones	Subtotal
1–2	inicializar R	$(M+1)+M$	$\Theta(M)$
3	inicializar S, enS	1	$\Theta(N)$
4	`para` t (cabeza)	$k+1$	$\Theta(k)$
5	$g_{\text{máx}}$, $mejor$	k	$\Theta(k)$
6–7	`para` i (cabeza), `si no` $enS[i]$	$k(N+1)+kN$	$\Theta(kN)$
8	$d\leftarrow 0$	P	$\Theta(kN)$
9–10	`para` j (cabeza) y acumulación	$P(M+1)+PM$	$\Theta(kNM)$
11–12	comparar y actualizar	$\le 2P$	$O(kN)$
13	agregar $mejor$	k	$\Theta(k)$
14–15	actualizar R	$k(M+1)+kM$	$\Theta(kM)$

El término dominante es la línea 10, ejecutada exactamente $P\cdot M$ veces: $T(N,k,M)\in\Theta(k\cdot N\cdot M)$. Esta clase es *invariante* para el **pseudocódigo**, porque el Algoritmo 3 no tiene parada temprana: siempre hace k pasos. El **código** (`select.py::voraz`) sí se detiene antes: cuando ningún candidato factible cabe en el costo o cuando la mayor ganancia es ≤ 0. Por eso su mejor caso es $\Theta(NM)$ y solo su peor caso es $\Theta(kNM)$ (Proposición 3.9). Una evaluación de ganancia marginal (líneas 8–10) cuesta $\tau=\Theta(M)$, de acuerdo con la forma $O(N\cdot k\cdot\tau)$ del enunciado. **En el JSON** ($M=N$): $T(N,k)\in\Theta(k\,N^2)$. Espacio: $\Theta(NM)$ por la matriz W (entrada) y $\Theta(N+M)$ adicional.

Nota de fidelidad al código (hallazgo del verifier, C-0008). El pseudocódigo omite los nodos ya elegidos (línea 7), de modo que sus evaluaciones útiles son $P=kN-k(k-1)/2$. La implementación (`src/ada_g2/select.py`) calcula en cada paso las ganancias de *todos* los candidatos con el producto matriz-vector $W(a\odot R)$ y después enmascara a los ya elegidos; por eso hace literalmente $k\cdot N\cdot M$ multiplicaciones y no $P\cdot M$. Como $P>kN/2$, la diferencia es un factor menor que 2 y la clase $\Theta(kNM)$ se mantiene. La prueba `test_conteo_evaluaciones_P_por_M` (`tests/test_seleccion.py`) cuenta las evaluaciones.

Fuerza bruta (referencia exacta). Enumera los $\sum_{r=0}^{k}\binom{N}{r}$ subconjuntos de tamaño $\le k$ y evalúa cada uno en $O(rM)$: $T_{\text{fb}}\in O\big(kM\sum_{r\le k}\binom{N}{r}\big)$; para $k\le N/2$, $\binom{N}{r}$ crece con r y $\sum_{r\le k}\binom{N}{r}\le(k+1)\binom{N}{k}$, exponencial en N si k es proporcional a N. En el modo `k` basta con los de tamaño exactamente k (Observación 1.8). *Fidelidad al código:* `exact.py::fuerza_bruta` recorre los subconjuntos en profundidad (DFS) y mantiene R de forma incremental, así que cada subconjunto visitado cuesta $\Theta(M)$ y no $O(rM)$. Como el DFS pasa por todos los prefijos, en el modo `k` visita también los subconjuntos de tamaño $<k$. La cota ajustada que resulta es $\Theta\big(N+M\sum_{r\le k}\binom{N}{r}\big)$ (Proposición 3.11).

3.3. Mejor caso, peor caso y caso promedio

Esta sección da, para cada algoritmo *tal como está implementado* en `src/ada_g2/`, T_{mejor}, T_{peor} y T_{prom}, con una instancia testigo para cada uno. Seguimos la distinción de mejor y peor caso de [2, §2.1.1] y la notación asintótica en varias variables de [2, cap. 1]. Escribimos Θ solo cuando tenemos cota superior e inferior; si no, usamos O u Ω y lo decimos.

Convenciones. log es $\log_2$. Escribimos $1+\log N$ para que las cotas valgan también con $N = 1$; si $N \geq 2$, $\Theta(N + B(1 + \log N)) = \Theta(N + B \log N)$. Las variables de tamaño son (N, B) en la Parte 1, (N, M, k) y, según el modo, L en la Parte 2, y T_r para el número de rondas (en el JSON $M = N$). Sea $|E|$ el número de aristas del JSON. Mantenemos el modelo de costo unitario de D-13. Además suponemos que el producto denso matriz-vector de NumPy, `W @ v` con $W \in \mathbb{R}^{N \times M}$, cuesta $\Theta(NM)$ operaciones *sin importar los valores* (no aprovecha los ceros). **Caso promedio**: las notas de clase definen T_{mejor} y T_{peor}; el caso promedio exige fijar una distribución sobre las entradas, y la declaramos en cada caso. Usamos dos modelos.

- **Modelo A** (Parte 1): los a_i son independientes y tienen una distribución continua en $[\alpha, \beta]$ con $\alpha > 0$; N, B y los topes r son fijos.
- **Modelo G** (Parte 2 y solución completa): es la distribución de `generator.py::generar`. Los pesos son $\text{round}(U(0.1, 0.9), 2) \in [0.1, 0.9]$ (nunca 0 ni 1; si `densidad`< 1, las aristas ausentes valen 0); los coeficientes son $\text{round}(U(5, 15), 1) \geq 5$; los costos son enteros uniformes en $\{2, \ldots, 7\}$; los topes son enteros en $\{10, \ldots, 24\}$; y, por defecto, $L = \text{round}(3.6k)$.

Las afirmaciones de conteo marcadas "(verifier)" están comprobadas en `tests/test_casos_complejidad.py`.

Lema 3.6 (costo de una extracción en `heapq` de CPython; leído en el código fuente)**.** *Sea $h \geq 1$ el número de elementos sobre los que se llama `_siftup(heap,0)`. Para `heapreplace`, $h = |H|$. Para `heappop`, $h = |H| - 1$, porque primero se retira el último elemento. Entonces el ciclo de descenso de `_siftup` itera d veces, con $\lfloor \log h \rfloor - 1 \leq d \leq \lfloor \log h \rfloor$, y el ascenso posterior (`_siftdown`) itera a lo sumo d veces. En consecuencia, la operación cuesta $\Theta(1 + \log h)$* para cualquier valor de las claves. *Si $h = 0$ (un `heappop` que vacía el montículo), no se llama a `_siftup` y el costo es $\Theta(1)$.*

Demostración. (1) *Fuente.* En `Lib/heapq.py` de Python 3.12 (líneas 137–161 y 260–278 del Python 3.12 instalado localmente; [11]), `heappop` y `heapreplace` colocan un elemento en la raíz y llaman a `_siftup(heap,0)`. Ese procedimiento "sube el hijo menor *hasta llegar a una hoja*" con `while childpos < endpos`, y solo después llama a `_siftdown`. La línea 583 importa la versión en C (`from _heapq import *`). Su `siftup` en `Modules/_heapqmodule.c` (CPython 3.12, consultado en el repositorio oficial [11]) tiene la misma estructura: `while (pos < limit)` y después `siftdown`. Así, el ciclo de descenso termina solo al llegar a una hoja; las claves solo deciden *qué* hijo sube. (2) En un árbol binario completo de h nodos guardado en un arreglo, la posición p tiene profundidad $\lfloor \log(p+1) \rfloor$. Las hojas son las posiciones $p \geq \lfloor h/2 \rfloor$, con profundidad $\lfloor \log(\lfloor h/2 \rfloor + 1) \rfloor \geq \lfloor \log h \rfloor - 1$, y ninguna posición es más profunda que $\lfloor \log h \rfloor$. Esto da las cotas de d. (3) El ascenso vuelve por el mismo camino, así que hace a lo sumo d pasos. Cada paso cuesta $\Theta(1)$, porque compara tuplas $(-g, i)$ de tamaño fijo. Sumando, el costo es $\Theta(1 + d) = \Theta(1 + \log h)$. ■

(Verifier) `heappop` sobre un montículo de $n \geq 2$ elementos hizo al menos $\lfloor \log(n-1) \rfloor$ comparaciones en todos los casos probados, con n de 3 a 300 y varios tipos de claves. Con $n = 2$ hace 0 (`test_heappop_*`). Esa prueba es empírica y acompaña a la demostración, pero no la reemplaza.

Proposición 3.7 (casos del reparto, `allocate.py::asignar`; demostrado por nosotros)**.** *Sean $J = \min(B, \sum_i r_i)$ y h_j el tamaño sobre el que opera la extracción j ($h_j \leq N' \leq N$). Entonces*

$$T_{asignar} = \Theta\Big(N + \sum_{j=1}^{J} (1 + \log h_j)\Big).$$

(a) $T_{mejor}(N, B) = \Theta(N)$. Testigo: $B = 0$, o $r \equiv 0$.
(b) $T_{peor}(N, B) = \Theta(N + B(1 + \log N))$. Testigo: $r_i > B$ para todo i.
(c) En el modelo A con $r_i > B$ para todo i: $T_{prom} = \Theta(N + B(1 + \log N)) = T_{peor}$. Con topes arbitrarios solo afirmamos $T_{prom} \in \Omega(N + J) \cap O(N + J(1 + \log N))$.

Demostración. Costeo del código (`allocate.py`, líneas 43–65):

- l. 44–45: crean `x` y la lista del montículo, $\Theta(N)$.
- l. 46: `heapify` hace $\lfloor N'/2 \rfloor$ llamadas a `_siftup`, que suman $\Theta(N')$ [4, cap. 6].
- l. 51–62: el ciclo se ejecuta exactamente J veces (Teorema 3.1; el verifier lo comprobó: `heappop`+`heapreplace`$= J$ en 405 instancias). Cada iteración hace $\Theta(1)$ de trabajo (ganancia, traza, incrementos) y una extracción de costo $\Theta(1 + \log h_j)$ (Lema 3.6).
- l. 64: suma N utilidades, $\Theta(N)$.

Sumando se obtiene la fórmula. (a) Si $J = 0$, la suma es vacía y $T = \Theta(N)$. Además $T \geq cN$ siempre, por las líneas 44 y 64. (b) *Cota superior*: $J \leq B$ y $h_j \leq N$. *Cota inferior*: si $r_i > B$, entonces $\sum_i r_i \geq B$ y $J = B$. Tras cualquier extracción, $x_i \leq B < r_i$, así que siempre se ejecuta `heapreplace` (línea 59), nunca `heappop`, y $h_j = N$ para todo j. Luego $T \geq c\,(N + B(1+\log N))$. (c) J no depende de a. Si $r_i > B$, el argumento de (b) da $h_j = N$ para *todo* vector a. Entonces $T(a)$ está entre dos múltiplos de $N + B(1 + \log N)$ que no dependen de a, y lo mismo vale para su esperanza. Con topes que se agotan, `heappop` encoge el montículo (una vez por cada nodo que agota su tope; verifier), y el perfil (h_j) depende de *cuáles* nodos se saturan primero, es decir, de a. Solo se sostiene la cota general $1 \leq 1 + \log h_j \leq 1 + \log N$. ■

Observación 3.8 (por qué hay que tratar los topes con cuidado). Con $r \equiv 1$ y $B \geq N$ se tiene $J = N$ y $h_j = N - j$ (cada extracción es un `heappop`). Entonces $T = \Theta(N + \sum_{j<N}(1 + \log(N - j))) = \Theta(N + \log N!) = \Theta(N \log N)$, *aunque B sea enorme.* Como T solo depende de J y no de B directamente, el peor caso en función de (N, B) exige topes holgados.

En las rondas el reparto se hace sobre S (`rounds.py`, l. 128). Con $N \leftarrow |S|$: $\Theta(|S| + J(1 + \log |S|))$. Con $|S| \leq k$ (modos `k` y `ambos`), el peor caso es $\Theta(k + B(1 + \log k))$. En el modo `costo`, $|S| \leq N$.

Referencia exacta del reparto (`exact.py::dp_reparto`). Para cada nodo i y cada $b \in \{0, \ldots, B\}$, el ciclo interno hace $\min(b, r_i) + 1$ iteraciones. Para $r \leq B$, $\sum_{b=0}^{B} \min(b, r) = \frac{r(r+1)}{2} + (B - r)r \in [\frac{Br}{2}, Br]$; para $r > B$ vale $\frac{B(B+1)}{2}$. Sumando y agregando la creación de las tablas, $\Theta(N(B+1))$, se obtiene $T_{\text{dp}} = \Theta\big(N(B+1) + B\sum_i \min(B, r_i)\big)$. El costo no depende de a, así que T_{prom} coincide con T en cualquier modelo que fije (N, B, r). $T_{\text{mejor}} = \Theta(N(B+1))$ con $r \equiv 0$, y $T_{\text{peor}} = \Theta(N(B+1)^2)$ con $r_i \geq B$. Espacio $\Theta(N(B+1))$.

Proposición 3.9 (casos del voraz, `select.py::voraz` y `seleccionar`; demostrado por nosotros)**.** *Sea I el número de veces que se ejecuta el cuerpo del ciclo de la línea 74. Es igual al número de llamadas a `ganancias_todas`. Sea $p = |S|$ al salir. Entonces $p \leq I \leq \min(k_{\text{máx}}, p+1)$ y*

$$T_{voraz} = \Theta(N + M + I \cdot NM), \qquad \text{que es } \Theta(I \cdot NM) \text{ si } k_{\text{máx}} \geq 1.$$

Recordamos que $k_{\text{máx}} = k$ en los modos `k` y `ambos`, y $k_{\text{máx}} = \#candidatos \leq N$ en el modo `costo`.

(a) $T_{mejor} = \Theta(NM)$ en los tres modos.

(b) $T_{peor} = \Theta(kNM)$ en los modos `k` y `ambos`. En el modo `costo`, $T_{peor} = \Theta(\min(N, L + 1)\, NM)$ si todos los $c_i \geq 1$, y $\Theta(N^2 M)$ si se permiten costos 0 (el validador acepta $c_i \geq 0$).

(c) En el modelo G con todos los nodos candidatos y $k < N$: $T_{prom} = \Theta(kNM)$ en los modos `k` y `ambos` (este último con el L por defecto). En el modo `costo`, $T_{prom} = \Theta(\min(N, L+1)NM)$ para $L \geq 7$.

`seleccionar` tiene las mismas clases. En el modo `k` ejecuta un solo voraz. En los modos `costo` y `ambos` ejecuta dos voraces y `mejor_singleton`; este último cuesta $\Theta(NM)$, porque calcula una vez `ganancias_todas`.

Demostración. Costeo (`select.py`, líneas 66–95):

- l. 67–69: $\Theta(N + M)$.

- Cada iteración del ciclo: l. 75 cuesta $\Theta(NM)$ (supuesto del producto denso); l. 76–85 cuestan $\Theta(N)$ (máscaras y `argmax`).
- Solo si se acepta el elemento: l. 90–91 arman la lista `evaluados`, $O(N)$, y l. 94 actualiza R, $\Theta(M)$.

La línea 75 va *antes* de las dos salidas (`break`) de las líneas 77 y 86, así que la iteración que sale también paga $\Theta(NM)$. Total: $\Theta(N+M) + I\,\Theta(NM+N) + p\,O(N+M)$. Como $p \le I$ y $N+M \le 2NM$, el total es $\Theta(N+M+I\cdot NM)$.

Cotas de I. Cada iteración agrega un elemento o sale. Por eso $I = p$ si el ciclo termina por la condición $|S| = k_{\text{máx}}$, e $I = p+1$ si termina por `break`. En ambos casos $I \le k_{\text{máx}}$.

(a) Si $k_{\text{máx}} \ge 1$, entonces $I \ge 1$, y por tanto $T = \Omega(NM)$. *Testigo 1* (modos `costo` y `ambos`): $c_i > L$ para todo i. En la primera iteración la máscara `fact` queda vacía; resultan $I = 1$ y $S = \emptyset$. *Testigo 2* (modo `k`, $k \ge 2$): un nodo "estrella" e con $w_{ej} = 1$ para todo j. Como $a_j > 0$ y $w \le 1$, la ganancia de cualquier i es $\sum_j a_j w_{ij} \le \sum_j a_j$, con igualdad si y solo si la fila i es toda de unos. El elegido tiene, entonces, una fila de unos, y la línea 94 deja $R \equiv 0$. En la segunda iteración todas las ganancias valen 0 y se sale por la línea 86: $I = 2$, $|S| = 1$. En ambos testigos, $T = \Theta(NM)$ (verifier: `test_voraz_mejor_caso_*`). En `seleccionar`, el testigo 1 da exactamente 3 llamadas: $1+1$ de los voraces y 1 del singleton.

(b) *Modos `k` y `ambos`:* $I \le k$ da $O(kNM)$. *Testigo*: $w_{ij} < 1$ para todo $i \ne j$, $k < N$ y, en el modo `ambos`, $L \ge \sum_i c_i$. Si $i \notin S$, entonces $\Delta(i \mid S) \ge a_i w_{ii} R_i = a_i \prod_{l\in S}(1 - w_{li}) > 0$, y siempre quedan candidatos factibles. Por lo tanto nunca se sale por `break`, y $p = I = k$. *Modo `costo`:* $I \le k_{\text{máx}} \le N$. Si $c_i \ge 1$, como cada elemento aceptado consume al menos una unidad de L, se tiene $p \le L$ e $I \le \min(N, L+1)$. *Testigo*: $c \equiv 1$ y $w_{ij} < 1$ fuera de la diagonal. Si $L \ge N$, se eligen todos ($I = N$). Si $L < N$, se eligen L y la iteración $L+1$ sale por `fact` vacía ($I = L+1$). Con $c \equiv 0$, $I = N$.

(c) En el modelo G, $1 - w_{li} \ge 0.1$ y $a_i \ge 5$, así que para $i \notin S$ se cumple $\Delta(i \mid S) \ge 5 \cdot 0.1^{|S|} > 0$. *Modo `k`*: el argumento de (b) da $I = k$ para *toda* instancia de G. El costo es invariante sobre G y su promedio es $\Theta(kNM)$ (verifier: exactamente $k_{\text{máx}}$ pasos en 60 instancias). *Modo `ambos`*: si el costo restante es al menos $7 \ge \max_i c_i$, todo candidato cabe. Luego $p \ge \min(k, \lfloor L/7 \rfloor)$ e $I \ge \max(1, \lfloor L/7 \rfloor)$. Con $L = \text{round}(3.6k)$ esto es $\Theta(k)$, y como $I \le k$, se obtiene $\Theta(kNM)$. El mismo argumento vale para el criterio de razón, porque también sale solo cuando no cabe nadie o cuando la ganancia es ≤ 0. *Modo `costo`*: $\lfloor L/7 \rfloor \le p \le \lfloor L/2 \rfloor$, siempre acotado por N. ■

Observación 3.10 (aritmética de punto flotante). En (c), la positividad se usa en doble precisión IEEE-754. $5 \cdot 0.1^{|S|}$ es representable mientras $|S|$ no sea del orden de 320; con k mayor, R_i podría desbordarse por abajo (*underflow*) hasta 0 y el voraz pararía antes. **[VERIFICAR]** (umbral exacto; irrelevante para los tamaños del proyecto).

Proposición 3.11 (casos de la fuerza bruta, `exact.py::fuerza_bruta`; demostrado por nosotros)**.** *Sean $n \le N$ el número de candidatos, $c \ge 0$ los costos, V el número de visitas (el contador `cuenta`), X el número de iteraciones del ciclo de la línea 54, y $V_{\text{máx}} = \sum_{r=0}^{\min(k_{\text{máx}},n)} \binom{n}{r}$.*

(i) Un subconjunto S se visita si y solo si $|S| \le k_{\text{máx}}$ y $c(S) \le L$.

(ii) $V - 1 \le X \le V_{\text{máx}} - 1$ y $T_{fb} = \Theta(N + M \cdot V + X)$.

(iii) $T_{mejor} = \Theta(N+M)$ en los modos `costo` y `ambos`. Testigo: $c_i > L$ para todo i (verifier: $V = 1$).

(iv) $T_{peor} = \Theta(N + M \cdot V_{\text{máx}})$. Testigo: $L \ge \sum_i c_i$ (verifier: $V = V_{\text{máx}}$, también con $k_{\text{máx}} \ge n$). Esto es $\Theta(N + M\sum_{r\le k}\binom{N}{r})$ en los modos `k` y `ambos`, y $\Theta(M\,2^N)$ en el modo `costo`.

(v) En el modo `k` el costo es invariante*: $T_{mejor} = T_{peor} = T_{prom} = \Theta(N + M\sum_{r\le k}\binom{N}{r})$.*

(vi) Caso promedio: $\mathbb{E}[V] = \sum_{|S|\le k_{\text{máx}}} \Pr[c(S) \le L]$ y $T_{prom} = \Theta(N + M\,\mathbb{E}[V] + \mathbb{E}[X])$.

Demostración. (i) El DFS con `inicio`=`pos`+1 genera cada $S = \{i_1 < \cdots < i_r\}$ a lo sumo una vez, por la cadena de sus prefijos. La rama se corta al agregar i_m si y solo si $c(\{i_1, \ldots, i_m\}) > L$ (línea 56), o deja de extenderse cuando el tamaño llega a $k_{\text{máx}}$ (línea 51). Como $c \ge 0$, los costos

de los prefijos no decrecen, así que todos los prefijos caben si y solo si $c(S) \le L$. (ii) Cada visita distinta de la raíz proviene de una iteración del ciclo, así que $V - 1 \le X$. Cada iteración del ciclo, podada o no, corresponde a un subconjunto distinto $S \cup \{i\}$ con $|S \cup \{i\}| \le k_{\text{máx}}$, así que $X \le V_{\text{máx}} - 1$. Costo por visita: $\Theta(1)$, más `aR` de costo $\Theta(M)$ si $|S| < k_{\text{máx}}$, más la creación de la visita (`W[i]@aR` y `R*(1-W[i])`, $\Theta(M)$; para la raíz, `np.ones`, $\Theta(M)$). Cada iteración podada cuesta $\Theta(1)$. El preámbulo cuesta $\Theta(N)$ (`carr`) más `sorted`, que es $\Theta(n)$ porque las listas que recibe ya vienen ordenadas (`rounds.py`, l. 117). `num_subconjuntos` hace $O(k_{\text{máx}})$ llamadas a `comb`, con costo unitario asumido (se usa con $N \le 20$). (iii) $V = 1$ y $X = n$: el costo es $\Theta(N + M + n) = \Theta(N + M)$. (iv) Sin poda, $V = V_{\text{máx}}$ y $X = V_{\text{máx}} - 1$. La cota superior sale de (ii), porque $V \le V_{\text{máx}}$. En el modo `costo`, $k_{\text{máx}} = n$ y $V_{\text{máx}} = 2^n$. (v) El modo `k` no poda por costo ($L = \infty$), así que vale (iv) para toda instancia. (vi) Por (i), $V = \sum_{|S| \le k_{\text{máx}}} \mathbf{1}[c(S) \le L]$; se toma esperanza. ■

Caso promedio en el modelo G (modo `ambos`). Como $c_i \in \{2, \dots, 7\}$, todo subconjunto con $|S| \le \lfloor L/7 \rfloor$ cabe y ninguno con $|S| > \lfloor L/2 \rfloor$ cabe. Por lo tanto, para *toda* instancia de G,

$$\sum_{r=0}^{\min(k,\lfloor L/7 \rfloor)} \binom{N}{r} \;\le\; V \;\le\; \sum_{r=0}^{\min(k,\lfloor L/2 \rfloor)} \binom{N}{r},$$

y lo mismo vale para $\mathbb{E}[V]$. No damos una fórmula cerrada de $\mathbb{E}[V]$, porque depende de la distribución de la suma de costos discretos uniformes. Se medirá en T13. Con $L = \Theta(k)$ y k proporcional a N, ambas cotas son exponenciales en N. Nota de implementación: si V supera `max_visitas`$= 3 \cdot 10^6$, la rutina aborta (`ExactoDemasiadoGrande`); el análisis anterior ignora ese tope.

Certificados (`certificates.py`). *Reparto* (`certificar_reparto`, sobre $n = |S|$ nodos): elegir λ cuesta $\Theta(1)$, porque es la ganancia de la última unidad de la traza, o 0. Si $B = 0$, se calcula un máximo sobre n nodos, $\Theta(n)$. La cota UB(λ) suma n veces `_max_entero`, que prueba a lo sumo 4 candidatos enteros de costo $\Theta(1)$. Total $\Theta(n)$ en todos los casos: es invariante y *no depende de B*. *Selección* (`certificar_seleccion`): reconstruir R cuesta $\Theta(M + |S|M)$; `ganancias_todas`, $\Theta(NM)$; armar `idx` y los datos del PL, $\Theta(N)$. A eso se suma C_{PL}, el costo de `linprog(method="highs")` sobre $n' \le N$ variables acotadas en $[0, 1]$ y a lo sumo 2 restricciones. Según la documentación de SciPy [12], `"highs"` "elige automáticamente entre `highs-ds` y `highs-ipm`" (símplex dual o punto interior). **No acotamos C_{PL}.** La programación lineal es resoluble en tiempo polinomial en general (Khachiyan, 1979 [13]; no está en la bibliografía del enunciado), pero el símplex no tiene una cota polinomial de peor caso conocida en general [**VERIFICAR**]. El certificado es una *verificación* que se suma al algoritmo y no forma parte de él. Observación: si solo hay restricción de cardinalidad, el PL se resuelve sumando las k mayores $\Delta(e \mid S)$, en $O(N \log N)$; no lo implementamos así.

Solución completa (`rounds.py::ejecutar`, T_r rondas, $M = N$). Sean I_t las llamadas a `ganancias_todas` en la ronda t, S_t la selección y $J_t = \min(B, \sum_{i \in S_t} \text{tope}_i)$.

- Leer el JSON cuesta $\Theta(N^2 + |E|)$: `np.eye` crea W en $\Theta(N^2)$ y cada arista se procesa en $\Theta(1)$. Si no hay aristas repetidas, $|E| \le N(N-1)/2$ y el total es $\Theta(N^2)$.
- Por ronda: `disp`, `mask`, `topes` y `rem_antes` cuestan $\Theta(N)$; la selección, $\Theta(I_t N^2)$ (Proposición 3.9); el reparto, $\Theta(|S_t| + J_t(1 + \log |S_t|))$; `sorted`, $O(|S_t| \log |S_t|)$; `verificar_ronda`, $O(|S_t|N)$, porque `i not in disp` recorre una lista; `f_directa`, $\Theta(M + |S_t|M)$.

Como $|S_t| \le I_t$ (cada elemento de S_t lo agregó una iteración de algún voraz), todos los términos con $|S_t|$ quedan dominados por $I_t N^2$. Así

$$T_{\text{sol}} = \Theta\Big(N^2 + |E| + \sum_{t=1}^{T_r} \big(I_t N^2 + J_t(1 + \log |S_t|)\big)\Big) \quad \text{(sin certificados)},$$

y con certificados (la opción por defecto) se suma $\sum_t(\Theta(N^2) + C_{\mathrm{PL},t})$. Tomando $1 \le I_t \le k$ y $J_t \le B$ (modos `k` y `ambos`):

- $T_{\mathrm{mejor}} = \Theta(N^2 + |E| + T_r N^2)$. $I_t \ge 1$ siempre: en el modo `k`, porque $k \ge 1$; en los otros dos, porque el singleton siempre calcula `ganancias_todas`. Testigo: $B = 0$ y $c_i > L$ para todo i (modo `ambos`), con $I_t = 3$ y $S_t = \emptyset$; o $B = 0$ con un nodo estrella (modo `k`), con $I_t = 2$.
- $T_{\mathrm{peor}} = \Theta(N^2 + |E| + T_r(kN^2 + B(1 + \log k)))$. Testigo: modo `k` (o `ambos` con $L \ge \sum c$), $w_{ij} < 1$ fuera de la diagonal, $k < N$, política `concentrado` y $R_i > T_r B$ para todo i. Entonces en la ronda t el restante de cada nodo es al menos $R_i - (t-1)B > B$: todos los nodos están disponibles, $I_t = k$, $J_t = B$ y el montículo tiene k elementos en cada extracción. En el modo `costo`, k se reemplaza por $\mathrm{mín}(N, L+1)$, o por N si hay costos 0.
- T_{prom} en el modelo G (modo `k`, $q = 0$, $N \ge T_r k$): en cada ronda se agotan a lo sumo k nodos, así que hay al menos $N - (t-1)k \ge k$ candidatos e $I_t = k$. Resulta $T_{\mathrm{prom}} = \Theta\big(N^2 + T_r kN^2 + \log k \cdot \mathbb{E}[\sum_t J_t]\big)$, con $0 \le \sum_t J_t \le T_r B$. Por lo tanto, $T_{\mathrm{prom}} \in \Omega(T_r kN^2) \cap O(T_r(kN^2 + B \log k))$. Con la hipótesis adicional $R_i > T_r B$, se vuelve $\Theta(T_r(kN^2 + B \log k))$.

El subcomando `run` llama además a `comparar_exacto`, que es exponencial (Proposición 3.11) pero solo se ejecuta si $N \le 20$. Por eso es una referencia acotada para pruebas y no entra en el análisis asintótico.

Algoritmo	T_{mejor}	T_{peor}	T_{prom}	$S(n)$	Instancia testigo
Reparto `asignar` (N, B)	$\Theta(N)$	$\Theta(N + B(1 + \log N))$	$\Theta(N + B(1 + \log N))$ (modelo A, $r_i > B$)	$\Theta(N + J)$	mejor: $B = 0$; peor y promedio: $r_i > B$ para todo i
Reparto en una ronda (sobre S)	$\Theta(1 + \lvert S\rvert)$	$\Theta(k + B(1 + \log k))$	igual al peor si $R_i > T_r B$	$\Theta(k + B)$	como arriba, con $N \leftarrow \lvert S\rvert \le k$
DP del reparto (referencia)	$\Theta(N(B+1))$	$\Theta(N(B+1)^2)$	invariante en a	$\Theta(N(B+1))$	mejor: $r \equiv 0$; peor: $r_i \ge B$
Voraz / `seleccionar`, modos `k` y `ambos`	$\Theta(NM)$	$\Theta(kNM)$	$\Theta(kNM)$ (modelo G)	$\Theta(NM)$ entrada $+ \Theta(N + M + kN)$	mejor: nodo estrella (`k`) o $c_i > L$ (`ambos`); peor: $w_{ij} < 1$ fuera de la diagonal y $L \ge \sum c$
Voraz / `seleccionar`, modo `costo`	$\Theta(NM)$	$\Theta(\mathrm{mín}(N, L+1)NM)$ si $c \ge 1$; $\Theta(N^2 M)$ si hay $c_i = 0$	$\Theta(\mathrm{mín}(N, L+1)NM)$ (modelo G, $L \ge 7$)	$\Theta(NM) + \Theta(N^2)$ traza	mejor: $c_i > L$; peor: $c \equiv 1$ ($c \equiv 0$) y $w_{ij} < 1$
Fuerza bruta (`k`)	invariante: $\Theta\big(N + M\sum_{r \le k}\binom{N}{r}\big)$			$\Theta(N + kM)$	cualquiera
Fuerza bruta (`costo`/`ambos`)	$\Theta(N + M)$	$\Theta(N + M\,V_{\mathrm{máx}})$	$\Theta(N + M\,\mathbb{E}[V] + \mathbb{E}[X])$, entre las cotas de G; se mide en T13	$\Theta(N + k_{\mathrm{máx}} M)$	mejor: $c_i > L$; peor: $L \ge \sum c$
Certificado del reparto	invariante: $\Theta(\lvert S\rvert)$, independiente de B			$\Theta(\lvert S\rvert)$	cualquiera
Certificado de la selección	$\Theta(NM + \lvert S\rvert M) + C_{\mathrm{PL}}$; C_{PL} no acotado por nosotros **[VERIFICAR]**			$\Theta(N + M)$+PL	—
Solución completa ($M = N$, `k`/`ambos`, sin cert.)	$\Theta(N^2 + \lvert E\rvert + T_r N^2)$	$\Theta(N^2 + \lvert E\rvert + T_r(kN^2 + B(1 + \log k)))$	$\Omega(T_r kN^2) \cap O(T_r(kN^2 + B \log k))$ (modelo G, $N \ge T_r k$)	$\Theta(N^2 + T_r(kN + B))$	mejor: $B = 0$ y $c_i > L$; peor: $w_{ij} < 1$, $R_i > T_r B$

Cuadro 3: Resumen de complejidad temporal y espacial, con instancia testigo, de la implementación en `src/ada_g2/`. Convención: $1 + \log$ cubre $N = 1$ y $k = 1$.

Espacio. La columna $S(n)$ cuenta la memoria auxiliar más la entrada cuando esta domina. Detalles: el voraz guarda en cada paso la lista `evaluados` ($O(N)$), de ahí el $\Theta(kN)$; la fuerza bruta tiene profundidad de recursión $\le k_{\mathrm{máx}} + 1$ y cada marco guarda R y `aR`, de $\Theta(M)$ cada

uno; la solución completa conserva, por ronda, `disp`, `rem_antes`, `topes` ($\Theta(N)$), las trazas de selección ($O(kN)$) y la del reparto ($O(B)$).

3.4. Cota $1 - 1/e$ del voraz (modo `k`)

Usamos la noción de algoritmo aproximado y de razón de aproximación del libro del curso [3, cap. 9].

Teorema 3.12 (Nemhauser, Wolsey y Fisher, 1978 [7]; la prueba que sigue es el argumento estándar, escrito por nosotros; la extensión a otras restricciones está en [8])**.** *Hipótesis: (H1)* $f : 2^V \to \mathbb{R}$ *con* $f(\emptyset) = 0$*; (H2)* f *monótona; (H3)* f *submodular; (H4) la restricción es solo de cardinalidad,* $|S| \le k$*, con* $1 \le k \le N$*. Sea* S_k *la salida del voraz que, en* k *pasos, agrega un elemento de máxima ganancia marginal (cualquier desempate) y sea* $\mathrm{OPT} = \max\{f(S) : |S| \le k\}$*. Entonces*

$$f(S_k) \;\ge\; \Big[1 - \Big(1 - \frac{1}{k}\Big)^k\Big]\mathrm{OPT} \;\ge\; \Big(1 - \frac{1}{e}\Big)\mathrm{OPT} \approx 0.632\,\mathrm{OPT}.$$

Demostración. Sea O un óptimo, $|O| \le k$; S_t el conjunto del voraz tras t pasos; $\delta_t = \mathrm{OPT} - f(S_t)$.

(1) $\mathrm{OPT} = f(O) \le f(O \cup S_t)$, por (H2).

(2) Sea $O \setminus S_t = \{o_1, \dots, o_r\}$, $r \le k$. Telescópica: $f(O \cup S_t) - f(S_t) = \sum_{m=1}^{r} \Delta(o_m \mid S_t \cup \{o_1, \dots, o_{m-1}\}) \le \sum_{m=1}^{r} \Delta(o_m \mid S_t)$, por (H3) (cada $o_m \notin S_t \cup \{o_1, .., o_{m-1}\} \supseteq S_t$).

(3) Cada $o_m \notin S_t$ era candidato en el paso $t+1$, así que $\Delta(o_m \mid S_t) \le f(S_{t+1}) - f(S_t)$ (elección voraz). Por (H2) esa diferencia es ≥ 0, luego $r \cdot (\cdot) \le k \cdot (\cdot)$.

(4) De (1)–(3): $\mathrm{OPT} \le f(S_t) + k\,[f(S_{t+1}) - f(S_t)]$, que se reescribe $\delta_{t+1} \le (1 - \frac{1}{k})\,\delta_t$.

(5) $\delta_0 = \mathrm{OPT} - f(\emptyset) = \mathrm{OPT}$ por (H1). Iterando k veces: $\delta_k \le (1 - \frac{1}{k})^k \mathrm{OPT}$, es decir $f(S_k) \ge [1 - (1 - \frac{1}{k})^k]\mathrm{OPT}$.

(6) Como $1 - z \le e^{-z}$ para todo z, $(1 - \frac{1}{k})^k \le e^{-1}$.

■

Aplicación a nuestra f (modo `k`). (H1)–(H3) son el Lema 1.5 (con $a_j > 0$ y $w_{ij} \in [0, 1]$, que el programa valida); (H4) es la restricción del modo `k`; el voraz de la Sección 3.2 hace exactamente k pasos de máxima ganancia (posible porque $k < N$). Por tanto **el Algoritmo 3 garantiza** $f(S) \ge (1 - 1/e)\mathrm{OPT}$ **en el modo `k`**. La verificación empírica contra fuerza bruta (familias sintéticas, porque la instancia de la profesora se satura, R-02) se resume en el párrafo siguiente, y la razón observada en el caso de la profesora está en la Sección 4.

Verificación empírica de la cota (medida por el verifier; origen: `tests/test_seleccion.py`, tarea T12). Sobre 3 familias sintéticas de 250 instancias cada una (3×250) y en el modo `k`, la razón mínima voraz/exacto medida fue 0.8885 (familia rala; cifra registrada en `agent/CHANGELOG.md`, entrada C-0008), y ninguna instancia quedó por debajo de $1 - (1 - 1/k)^k$, que es lo que comprueba `test_restricciones_razon_y_certificado`. Sobre la instancia ajustada de D-15 (construida por `instancia_ajustada` en `src/ada_g2/generator.py`, reproducible con `python -m ada_g2 ajustada` y verificada en `test_instancia_ajustada_razon_exacta`) las razones son exactamente 3/4, 19/27 y 2101/3125 para $k = 2, 3, 5$, es decir $1 - (1 - 1/k)^k$ en cada caso: la cota es ajustada para este voraz. En los modos `costo` y `ambos` las razones mínimas medidas fueron 0.8864 y 0.8148 (misma fuente), **solo como medición y sin garantía teórica** (R-01).

Observación 3.13 (qué pasa si falla cada hipótesis). **(H1)**: si $f(\emptyset) = c > 0$, el mismo argumento da la cota para $f - c$, no para f. **(H3)**: con $V = \{1, 2, 3\}$, $f(S) = 10 \cdot \mathbf{1}[\{1, 2\} \subseteq S] + \mathbf{1}[3 \in S]$ (monótona, no submodular: $\Delta(2 \mid \emptyset) = 0 < 10 = \Delta(2 \mid \{1\})$) y $k = 2$, el voraz elige 3 (ganancia 1) y luego 1 (ganancia 0): $f = 1$, mientras OPT $= f(\{1, 2\}) = 10$; razón $0.1 < 0.632$. **(H4)**: con un presupuesto de costos la cota no aplica; con dos elementos de cobertura disjunta, $f(\{1\}) = 2$, $c_1 = 1$ y $f(\{2\}) = L$, $c_2 = L$ (con $L \geq 3$), el voraz por razón ganancia/costo elige 1 (razón $2 > 1$) y ya no cabe 2: obtiene 2 frente a OPT $= L$, razón $2/L \to 0$. Por esto la variante con costos usa el mejor entre el voraz por razón y el mejor nodo individual [10], y **para el modo `ambos` (cupo y costo a la vez) no afirmamos ninguna garantía teórica** (R-01): se mide la razón contra el exacto (Sección 4 y el párrafo de verificación empírica de la Sección 3.4).

Contraejemplo 3.14 (el voraz no es óptimo en la Parte 2)**.** $U = \{1, \ldots, 6\}$, $S_1 = \{1, 2, 3, 4\}$, $S_2 = \{1, 2, 5\}$, $S_3 = \{3, 4, 6\}$, $k = 2$, con la f de cobertura de la reducción. Paso 1: ganancias $4, 3, 3$, elige S_1. Paso 2: $\Delta(S_2 \mid \{S_1\}) = |\{5\}| = 1$ y $\Delta(S_3 \mid \{S_1\}) = |\{6\}| = 1$; empate, elige S_2. Voraz: $f = 5$. Óptimo: $S_2 \cup S_3 = U$, $f = 6$. Razón $5/6 \approx 0.833 \geq 1 - (1 - \frac{1}{2})^2 = 0.75$, como garantiza el Teorema 3.12. El voraz de la Parte 2 es aproximado, no exacto; el de la Parte 1 sí es exacto (Teorema 3.1).

Observación 3.15 (no se puede mejorar en general). Feige [9] demostró que, para Cobertura Máxima, ningún algoritmo polinomial garantiza una razón $1 - 1/e + \varepsilon$ para ningún $\varepsilon > 0$, salvo que P=NP (verificado a nivel de resumen de la publicación). Como Cobertura Máxima es un caso particular de nuestra Parte 2 (Lema 2.6), la cota del voraz es, en ese sentido, la mejor posible para algoritmos polinomiales.

4. Ejecución del caso de la profesora

Esta sección reporta lo que produjo el código sobre `project-docs/entrada_G2_red_15_nodos.json` con `python -m ada_g2 run`. **Nada de lo que sigue se calculó a mano**: cada cifra se toma de uno de los cuatro archivos de `results/caso_entrada_G2_red_15_nodos/` (`resultado.json`, `tiempo.csv`, `traza.md`, `traza.csv`) y la fuente se indica junto a cada tabla. Salvo que se diga otra cosa, las cifras reales se muestran con cuatro decimales. Parámetros de la corrida (`resultado.json`, campo `parametros`): $N = 15$, $T = 3$, $k = 5$, $B = 30$, $L = 24$, 105 aristas, modo `ambos` (cupo k y costo L a la vez, D-03), $q = 0$ (D-07), política de gasto `concentrado` (D-22), semilla 20261105. Entorno (`resultado.json`, campo `entorno`): Python 3.12.10, Windows-11-10.0.26300-SP0. El campo `avisos` está vacío y `todo_cumple` vale `true`.

4.1. Resultado por ronda

Ronda	Selección S_t	$f(S_t)$	$\sum_{i \in S_t} c_i$	Reparto x	Topes efectivos	$\sum_i u_i(x_i)$
1	N09, N11, N13, N14, N15	145.5795	23	7/5/5/7/6	18/18/18/18/18	88.5640
2	N09, N11, N13, N14, N15	145.5795	23	7/5/5/7/6	11/13/13/11/12	88.5640
3	N09, N11, N13, N14, N15	145.5795	23	4/8/8/4/6	4/8/8/4/6	85.6108
Totales		436.7386				262.7387

Cuadro 4: Resultado por ronda. Reparto y topes en el orden N09/N11/N13/N14/N15; la fila Totales es $\sum_t f(S_t)$ y $\sum_t \sum_i u_i(x_i)$. Origen: `resultado.json`, campos `rondas[].seleccion`, `f`, `costo_participacion`, `reparto`, `topes_efectivos`, `utilidad` y `totales.suma_f`, `totales.suma_utilidad`.

Lectura. (i) Con $q = 0$ y ningún nodo agotado, el criterio de selección no cambia entre rondas (D-21): $S_1 = S_2 = S_3$. Solo cambia el reparto, porque los topes restantes $r_i(t)$ bajan (D-06). (ii) Los topes *sí* son activos en la ronda 3: tras las rondas 1 y 2 cada participante ha gastado $7 + 7 = 14$, $5 + 5 = 10$, $5 + 5 = 10$, $7 + 7 = 14$ y $6 + 6 = 12$ unidades, le quedan $4, 8, 8, 4, 6$ (suman $30 = B$) y el reparto de la ronda 3 coincide con esos topes. En esa ronda no hay nada que optimizar sobre la repartición: toda la capacidad restante se usa. (iii) Cada uno de los cinco participantes termina con gasto total 18, igual a su `presupuesto_total_nodo` (`resultado.json`, `totales.gasto_por_nodo` y `totales.presupuesto_total_nodo`).

4.2. Traza resumida de la ronda 1

Selección (Parte 2), heurística elegida: `voraz_ganancia`. Origen: `traza.md`, sección «Ronda 1 / Selección», con seis decimales como allí. La última columna da el mejor competidor en ese paso, es decir, el segundo mayor de las ganancias marginales de los candidatos factibles (columna «ganancias de los candidatos factibles» de `traza.md`).

Paso	Elegido	Ganancia marginal $\Delta(i \mid S)$	Costo acum.	f acum.	Mejor competidor
1	N11	103.490000	4	103.490000	N09 (95.7000)
2	N14	24.278250	7	127.768250	N09 (23.3850)
3	N09	9.932737	10	137.700987	N07 (9.3122)
4	N15	5.530613	16	143.231600	N13 (4.1325)
5	N13	2.347920	23	145.579520	N12 (2.2442)

Cuadro 5: Las cinco elecciones del voraz en la ronda 1. Las ganancias decrecen paso a paso (submodularidad, Lema 1.5) y la suma de las ganancias marginales es $f(S)$.

Las otras dos heurísticas del modo `ambos` dieron (`resultado.json`, `rondas[0].detalle_seleccion.alternativas`): `voraz_razon`, $S = \{\text{N05, N09, N14, N11, N15}\}$ con $f = 145.1056$ y costo 18; y `singleton`, $S = \{\text{N11}\}$ con $f = 103.4900$ y costo 4. Se elige la de mayor f (empate: la primera en el orden ganancia, razón, singleton; D-23). El voraz por ganancia termina con $k = 5$ nodos elegidos (cupo agotado) y costo $23 \leq 24$.

Reparto (Parte 1), unidad a unidad. Participantes N09, N11, N13, N14, N15; tope de cada uno 18. Origen: `traza.md`, sección «Reparto (Parte 1), unidad a unidad».

Unidad	Nodo	x antes	Ganancia marginal $g_i(x)$	Utilidad acum.
1	N14	0	7.000787	7.000787
2	N09	0	6.862157	13.862944
3	N15	0	6.584898	20.447842
4	N13	0	5.753122	26.200963
5	N11	0	5.267919	31.468882
⋮				
30	N09	6	1.321961	88.563953

Cuadro 6: Primeras cinco unidades y última unidad del reparto de la ronda 1. La última ganancia, 1.3220, es el multiplicador λ del certificado de dualidad (Sección 4.4).

El reparto final es N09= 7, N11= 5, N13= 5, N14= 7, N15= 6 con utilidad 88.563953. El montículo saca siempre la mayor ganancia marginal disponible; como $g_i(x) = a_i \ln \frac{x+2}{x+1}$ es decreciente en x, las 30 unidades tomadas son las 30 mayores ganancias del multiconjunto (Teorema 3.1).

4.3. Verificación de restricciones

El código comprueba explícitamente, por ronda, las siete restricciones y la coherencia numérica de la Tabla 7, y al final la restricción global del presupuesto total por nodo. Origen: `resultado.json`, campo `rondas[].verificacion_restricciones` y `verificacion_global`; la columna «valor» corresponde a la ronda 1.

Restricción	Significado	Valor	Límite	Cumple
$\|S\| \leq k$	cupo de participantes	5	5	sí
$\sum c \leq L$	costo de participación	23	24	sí
$\sum x \leq B$	presupuesto de ruido de la ronda	30	30	sí
$x_i \leq \text{tope}_i$	tope por nodo (política)	-11	0	sí
$x_i \leq \text{restante}_i$	presupuesto total restante	(sin violaciones)	(ninguna)	sí
$x_i \geq 0$ entero	dominio	5 (mínimo de x)	0	sí
$S \subseteq$ disponibles	participantes disponibles	(sin violaciones)	(ninguna)	sí
f incremental $= f$ directa	coherencia numérica	2.8422×10^{-14}	10^{-9}	sí

Cuadro 7: Restricciones verificadas en la ronda 1. En las rondas 2 y 3 todas también cumplen; en la ronda 3 el valor de «tope por nodo» es 0 (holgura nula: el tope es activo) y el mínimo de x es 4. Restricción global $\sum_t x_{i,t} \leq R_i$: sin violaciones; `todo_cumple=true`.

En la fila de tope por nodo, «valor» es el máximo de $x_i - \text{tope}_i$ entre los participantes: -11 significa que el nodo más cercano a su tope todavía tiene 11 unidades de holgura.

4.4. Certificados y comparación con el exacto

Certificados por ejecución (D-14; demostrados en el código, no asumidos). Origen: `resultado.json`, `certificado_reparto` y `certificado_seleccion`.

- *Parte 1, cota de dualidad de Lagrange.* Rondas 1 y 2: $\lambda = 1.3220$; ronda 3: $\lambda = 0.8952$. En las tres rondas la cota superior coincide con la utilidad obtenida (88.5640, 88.5640 y 85.6108) y la **brecha de dualidad es** 0 (tolerancia 10^{-9}): el reparto es óptimo sin necesidad de conocer el óptimo por otro método.

- *Parte 2, cota a posteriori por programación lineal.* En las tres rondas $f(S) = 145.5795$, el programa lineal acota lo que podría añadir cualquier otro conjunto factible en 6.4076, y la **cota superior del óptimo es** 151.9871. La **razón certificada es** 0.9578: $f(S) \geq 0.9578 \cdot \text{OPT}$, con las restricciones de cupo y costo a la vez. Este certificado vale aunque el modo `ambos` no tenga garantía a priori (R-01).

Comparación con el exacto (referencia de validación, no parte del algoritmo). Origen: `resultado.json`, `rondas[].comparacion_exacto`.

- *Modo `ambos` (el de la ejecución).* La fuerza bruta evalúa 3 893 subconjuntos factibles de los 4 944 posibles de tamaño ≤ 5 (contando el vacío; 4 943 no vacíos, como en la Sección 1.5). El óptimo es $S^* = \{\text{N09, N11, N13, N14, N15}\}$ con $f^* = 145.5795$, el mismo conjunto del voraz: **razón voraz/óptimo** = 1.0000. En las tres rondas el reparto del voraz coincide con el de la programación dinámica: 88.5640, 88.5640 y 85.6108 (diferencia absoluta 1.4211×10^{-14}, `reparto_optimo=true`).

- *Modo `k` (solo cupo, sin costo).* Cifras de `tests/test_caso_profesora.py` (reproducibles con `-modo k`; no forman parte de los archivos de `results/` porque la corrida entregada es en modo `ambos`): el óptimo con $|S| \leq 5$ es $S^* = \{\text{N09, N11, N12, N13, N15}\}$ con $f^* = 145.9072$, explorando 4 944 subconjuntos, y el voraz ($f = 145.5795$) logra la razón 0.99775

($= 145.5795/145.9072$). La garantía teórica en este modo es $1-(1-1/5)^5 = 2101/3125 = 0.67232 \geq 1-1/e \approx 0.6321$ (Teorema 3.12); la razón observada la supera con holgura.

Una razón de 1.0000 en un único caso **no es una garantía**: es una medición. La garantía es la del Teorema 3.12, solo en el modo `k`.

4.5. Tiempos de ejecución

Medición con `time.perf_counter`, un calentamiento sin medir y 5 repeticiones (D-11); se reporta la mediana. Origen: `tiempo.csv` (columna `mediana_s`, convertida a milisegundos sin redondeo adicional).

Fase	Mediana (ms)	Fase	Mediana (ms)
`carga_json`	0.5687	`seleccion_r1`	0.4073
`seleccion` (3 rondas)	1.2734	`reparto_r1`	0.0433
`reparto` (3 rondas)	0.1350	`seleccion_r2`	0.4279
`verificacion`	0.0526	`reparto_r2`	0.0416
`certificados`	5.2023	`seleccion_r3`	0.3949
`pipeline_total`	6.6853	`reparto_r3`	0.0484
`referencia_exacta` (1 sola corrida, fuerza bruta + DP de las 3 rondas): 51.2363 ms			

Cuadro 8: Tiempos medianos del caso de la profesora. Los certificados (programa lineal incluido) dominan el pipeline; el algoritmo de selección y de reparto, sin certificados, cuesta $1.2734 + 0.1350 = 1.4084$ ms.

Con una sola instancia ($N = 15$) estos tiempos no permiten estimar la curva $T(n)$; esa estimación requiere el barrido por tamaños de la validación experimental (T13), que aún no se ha ejecutado. Se incluyen aquí solo como evidencia de que el caso se resuelve en pocos milisegundos.

4.6. Advertencia de saturación (R-02)

La instancia de la profesora es un grafo completo con pesos ≥ 0.2. Como $f(S) = \sum_j a_j[1 - \prod_{i\in S}(1-w_{ij})]$ y cada nodo queda cubierto con probabilidad cercana a 1 por cualquier conjunto de cinco, f **se satura**: $f(V) = \sum_j a_j = 147.8$ (suma de `coeficiente_utilidad` en `project-docs/entrada_G2_red_15_nodos.json`) y la selección de la ronda 1 alcanza $145.5795/147.8 = 0.9850$ de ese máximo. Consecuencias que conviene decir con claridad:

- Las tres heurísticas del modo `ambos` dan valores cercanos (`voraz_razon`: 145.1056, diferencia 0.4740 frente al voraz por ganancia), y el óptimo coincide con el voraz. El caso **discrimina poco** entre selecciones.
- Por eso este caso no es evidencia fuerte de la calidad de la aproximación. La verificación de la cota $1-1/e$ se hace sobre familias sintéticas (Sección 3.4, párrafo de verificación empírica) y sobre la instancia ajustada de D-15, no sobre este grafo.

Referencias

[1] L. E. Guerrero. *Proyecto Semestral, Análisis y Diseño de Algoritmos 2026-B*, cap. 3 (Subproyecto 2), pp. 10–12, y Proposición 4.3.1, p. 14. Universidad de Caldas, 2026.

[2] *Análisis y Diseño de Algoritmos — Notas de clase 2024-A* (§2.1 costeo, §2.3 invariantes, §3.1 montículos). Universidad de Caldas, 2024.

[3] E. Villegas Jaramillo y L. Guerrero Mendieta. *Análisis y Diseño de Algoritmos: un enfoque práctico*. Editorial Universidad Nacional, 2016.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Cap. 6 (montículos; construcción en tiempo lineal).

[5] R. M. Karp. Reducibility among combinatorial problems. En R. E. Miller y J. W. Thatcher (eds.), *Complexity of Computer Computations*, pp. 85–103. Plenum Press, 1972. (Lista de los 21 problemas comprobada con fuentes secundarias; texto original sin consultar [**VERIFICAR**].)

[6] M. R. Garey y D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979. [**VERIFICAR**] (capítulo y página de la dureza fuerte).

[7] G. L. Nemhauser, L. A. Wolsey y M. L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14(1):265–294, 1978.

[8] M. L. Fisher, G. L. Nemhauser y L. A. Wolsey. An analysis of approximations for maximizing submodular set functions—II. *Mathematical Programming Study*, 8:73–87, 1978.

[9] U. Feige. A threshold of $\ln n$ for approximating set cover. *Journal of the ACM*, 45(4):634–652, 1998.

[10] S. Khuller, A. Moss y J. Naor. The budgeted maximum coverage problem. *Information Processing Letters*, 70(1):39–45, 1999.

[11] Python Software Foundation. *CPython 3.12*: módulos `Lib/heapq.py` y `Modules/_heapqmodule.c`. `https://github.com/python/cpython/tree/3.12`. Consultado el 6 de noviembre de 2026.

[12] The SciPy community. `scipy.optimize.linprog`, documentación de referencia. `https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html`. Consultado el 6 de noviembre de 2026.

[13] L. G. Khachiyan. A polynomial algorithm in linear programming. *Soviet Mathematics Doklady*, 20:191–194, 1979. [**VERIFICAR**]